Record of Modification

Version	Date	Modification record
V2.0	2011-07-06	
V3.0	2011-09-22	
V4.1	2012-04-17	USB Image Download Function is added.
V4.2	2012-04-27	Virtual Keyboard Input bits is extended to 32 bytes (display character size is 24 dot matrix and below)
V4.8	2012-12-01	The clock(RTC) display model is added, and the speed is improved. The temporary buffer reset function is added. The speed of images\texts display is improved. The function of USB directly writing video memory is added.
V6.0	2013-01-01	Delete the function of UART port downloading configuration file(the updated version adopt USB mini port and SD card downloading). The CRC check function is added. The function of SD card downloading configuration file is added (support only for 2G-TF card). The custom clock display format is added. The command of displaying any small image is added. The command of cutting and displaying any large image is added. The display terminal with RS485 interface adds the 0x01 command for searching, reading, touching the keyboard, batch processing.
V6.2	2014-03-01	Delete the 0xE0 for working mode from UART command sets; the working mode is set by USB mini port. The function of checking, displaying download results and reporting error information is added when SD card is batch downloading offline. An optional handshake command 0x00 is added before the touch calibration command 0xE4(set by USB mini port) The keyboard control is added to achieve the input of values or characters for external keyboard. The function of screen protection is added, the backlight will automatically delay the turn off or dim (set by USB mini port). The boot notification command 00 0a of the display terminal is added(set by USB mini port). The command of line display with specified color 0x6A01 is added. The “PC development and demonstration assistant” was divided into three tools: "USB mini port work mode configuration and text download V6.2, serial debug tool V6.2 and hardware update tool V2.0 "

Content

1 Serial Port Description ..... - 5 -

1.1 Working Mode ..... - 5 -
1.2 Data Frame Structure ..... - 5 -
1.3 Communication Frame Buffer（FIFO） ..... - 5 -
1.4 Byte Transmission Sequence ..... - 5 -
1.5 Transmission Direction ..... - 5-

2. Command LIST ..... - 6 -

3. Command Description ..... - 7 -

3.1 Handshake(0x00) ..... - 7 -
3.2 Set the current color palette (0x40) ..... - 8 -
3.3 Set character spacing(0x41) ..... - 9-
3.4 Access specified position's color(0x42,0x43) ..... - 9 -
3.5 Cursor display (0x44) ..... - 9 -
3.6 Text display(0x54, 0x55, 0x6E, 0x6F,0x98,0x45) ..... - 10 -
3.7 Point display(0x50,0x51,0x74,0x72) ..... - 12 -
3.7.1 Set point（0x50，0x51） ..... - 12 -
3.7.2 Dynamic curve display (0x74) ..... - 12 -
3.7.3 Direct video memory operation (0x72) ..... - 13 -
3.8 Line display (0x56, 0x5D，0x75，0x76) ..... - 13 -
3.8.1 Connecting specified points（0x56,0x5D） ..... - 13 -
3.8.2 Connecting points with specified color (0x6A01) ..... - 13 -
3.8.3 Display circle with specified color (0x6a02) ..... - 13 -
3.8.4 Fill the circular domain with specified color(0x6a03) ..... -14-
3.8.5 Ellipse display（0x58） ..... -14-
3.8.6 Spectrum Display（0x75） ..... -14-
3.8.7 Broken Line Display（0x76） ..... -15-
3.8.8 Connecting points according to offset values (0x78) ..... -15-
3.9 Arc curve display(0x57) ..... - 15 -
3.9.1 Arc and circular domain display（0x57） ..... - 15 -
3.9.2 Arc segment display（0x5704） ..... - 15 -
3.10 Area display ..... - 16 -
3.10.1 Rectangle box or rectangle domain display(0x59,0x69,0x5A,0x5B,0x5C) ..... - 16 -
3.10.2 Area Fill(0x64) ..... - 16-
3.11 Clear screen（0x52） ..... - 17 -
3.12 Area translation (0x60,0x61,0x62，0x63) ..... - 17-
3.13 Image or icon display（0x70,0x71，0x99，0xE2，0x7B） ..... - 18 -
3.13.1 Image display（0x70） ..... - 18 -
3.13.2 Arbitrary small format image display ..... - 18 -
3.13.3 Arbitrary large format images displayed by cutting(0x70 02) ..... - 18 -
3.13.4 Display one image and figure up (0x7B) ..... - 19 -
3.13.5 Cut Icon(0x71、0x9C、0x9D) ..... - 19 -
3.13.6 User-defined Icon Display（0x99） ..... -20-
3.13.7Save current screen image in HMI（0xE2） ..... -20-
3.13.8 Refresh the full screen for once compulsorily (0xD0) ..... -20-
3.14 Backlight&brightness control（0x5E, 0x5F） ..... - 20 -

3.14.1 Backlight close（0x5E） ..... - 20 -
3.14.2 Commands start screen saver (0x5E) ..... - 20 -
3.14.3 turn the backlight to maxim brightness（0x5F） ..... -20-
3.14.4 Adjust backlight brightness（0x5F） ..... - 20 -
3.15 Touch panel operations (0x72,0x73,0x78,0x79,0xE4) ..... - 20 -
3.15.1 Automatically upload the touch coordinate (0x72,0x73) ..... - 20 -
3.15.2 Automatically upload touch key code(0x78) ..... - 21-
3.15.3Read the touch batch number (0x01, only for the RS-485 interface display terminal) ..... -21-
3.15.4 Touch panel calibration（0xE4) ..... - 21 -
3.16 Working mode configuration（0xE0） ..... - 21 -
3.17 Execute commands periodically and circularly（0x9A） ..... - 22 -
3.17.1 Turn on the function of executing commands periodically and circularly ..... - 22-
3.17.2 Turn off the function of executing commands periodically and circularly. ..... - 22 -
3.17.3 Automatic Cycle control configuration file ..... - 22 -
3.18 Temporary buffer operation（0xC0,0xC1,0XC2） ..... - 22 -
3.18.1 Write in the temporary buffer（0xC0） ..... - 23 -
3.18.2 Read from the temporary buffer(0xc2) ..... - 23 -
3.18.3 Set pointS using data in temporary buffer（0xC101） ..... - 23-
3.18.4 Connect lines using data in temporary buffer（0xC102） ..... - 23 -
3.18.5 Display line chart using data in temporary buffer（0xC103） ..... - 23 -
3.18.6 Display line chart at high speed using data in temporary buffer（0xC104） ..... - 24 -
3.18.7 Scal up or down line chart using data in temporary buffer（0xC105） ..... - 24 -
3.18.8 Display window limit two-way line chart in scale using data in temporary buffer (0xC106) ..... - 24 -
3.18.9 Buffer command to realize display synchronously using temporary buffer(0xC110) ..... - 25 -
3.19 Keyboard and I/O port operation ..... - 25 -
3.19.1 Upload key code（0x70） ..... - 25 -
3.19.2 Upload key code (0x71) ..... - 25 -
3.19.3 I/O port setting(0x34) ..... - 25 -
3.19.4 Write I/O port command(0x35) ..... - 25 -
3.19.5 Read I/O port command(0x36) ..... - 25 -
3.20 Reading and Writing in User's Memory（0x90,0x91） ..... - 27 -
3.20.1 Write in random data memory（0x90 64KB） ..... - 27 -
3.20.2 Write in sequence data memory（0x90 30MB） ..... - 28 -
3.20.3 Read data memory（0x91） ..... - 28 -
3.20.4 Copy the 64K random data to the sequential storage area (Note: Only the V6.0 and its later versions support this command) ..... -28-
3.20. 5 Write backup memory (0x93) ..... -28-
3.20.6 Read backup register (0x94) ..... -28-
3.21 Virtual Keyboard (0x7C) ..... - 29 -
3.21.1 variables or password input ..... -29-
3.21.2 Chinese, English, Number Input ..... -30-
3.21.3 Compulsively quit the virtual keyboard control ..... -31-
3.21.3 Calculator ..... -31-
3.22 Menu & List (0x7D) ..... - 31 -
3.23 Buzzer control（0x79） ..... - 33 -
3.24 Clock(RTC) Display and Read off（0x9B,0xE7） ..... - 34 -
3.24.1 Turn off clock display ..... - 34 -
3.24.2 Turn on clock display ..... - 34 -

3.24.3 Adjust clock ..... - 34 -
3.24.4 Read off current time (the Gregorian calendar) ..... - 34 -
3.24.5 Open Timer ..... - 35 -
3.24.6 Close Timer ..... - 35 -
3.25 Configuration File (Touchpad interface transfer) ..... - 35 -
3.26 Variable Display ..... - 38 -
3.26.1 Display integer Variable (0x1A 01) ..... - 38 -
3.26.2 Display float Variable (0x1A 02) ..... - 39-
3.26.3 Variable Control File display (0x1A 03) ..... - 40 -
3.27Dial Control ..... -43-
3.27.1Dial Control Design ..... -43-
3.27.2Download the dial control files ..... -44-
3.27.3 Dial control commands(0x9E) ..... -44-
3.27.4 Storage position of dial control ..... -45-
3.28 Download character library and configuration files（0xF2） ..... - 45 -
3.29 CRC checking code generating functions ..... - 46-

1 Serial port description

1.1 Working mode of serial port

Asynchronous, UART serial port is applied in our intelligent TFT-LCD Module with serial port mode of 8n1,i.e. 10 bits for every data transmission: one start bit, 8 data bits(LSB first) and one stop bit.

When using the USB port, the Assistant software will appear “USBmini download mode” icon, and the downloading speed is up to 12 M bps baud rate.

1.2 Data Frame Structure

The standard data frame consists of 4 data blocks, as shown in the table below:

Data Block	1	2	3	4
Examples	0xAA	0x70	0x00 0x01	0xCC 0x33 0xC3 0x3C
Description	header，fixed as 0xAA	command	Data, maximum 508 bytes	frame end

1.3 Communication Frame Buffer（FIFO）

There is a 24 Rx frame communication frame buffer with FIFO structure. Therefore, users can continuously send data to our module as long as the Rx frame doesn’t overflow.

The display module has a hardware pin (namly /Busy in user’s interface) to indicate the FIFO status. Normally, /Busy pin keeps high (RS232 is negative); when there is only one frame buffer area left, /Busy pin will go to low level.(RS232 is positive)

For general applications, as the processing speed is very fast, it is not necessary for the users to judge the status of /Busy pin.

However, for applications of sending multiple date frame in a short time, such as to rapidly refresh hundreds of monitor parameters at the same time, users are advised to use /Busy signal for controlling serial port; and when /Busy signal is low, stop sending data to HMI.

If "drop-frame" phenomenon (some data do not display) occurs in the process of using display module, overflowing of buffers may be the reason. Oscilloscope can help to inspect whether Busy signal alters. If so, slow down the transmitting speed or add hardware inspection to deal with Busy signal.

1.4 Byte Transmission Sequence

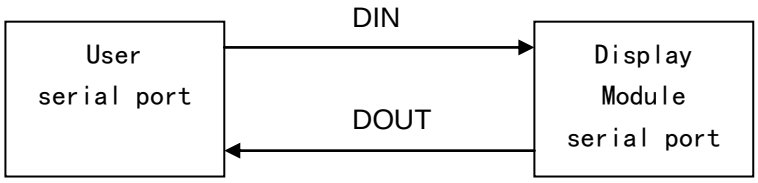
The format of all commands is Hex. For typestyle data(2 bytes), it is transmitted by as per MSB.

For example, X coordinate is 100, whose Hex format is 0x0064, then when x is transmitted, the proper sequence shall be 0x00 0x64.

1.5 Transmission Direction

Transmission direction is defined as follows:

- Downstream（Tx） when user sends data to display module, data is input via DIN pin;
- Upstream（Rx） when display module sends data to user, data is output from DOUT pin.



2. Command Table

Type	Comma	Parameters	Description
Handshaking	0x00	none	view configurations and version information
Display parameter setting	0x40	Fcolor+Bcolor	set up the palette
	0x41	D_X (0x00-0x7F) +D_Y (0x00-0x7F)	set up characters' display pitch
	0x42	X+Y	take color to background palette
	0x43	X+Y	take color to foreground palette
	0x44	Mode+X+Y+Wide(0x01-0x1F)+Height(0x01-0x1F)	set up the cursor display mode
Text display	0x53	X+Y+String	8x8 dot matrix ASCII character
	0x54		16x16 dot matrix, GBK extended code string
	0x55		32x32 dot matrix, GB2312 internal code string
	0x6E		12x12 dot matrix, GBK extended code string
	0x6F		24x2 dot matrix, GB2312 internal code string
	0x98	X+Y+Lib_ID+C_mode+C_dot+Fcolor+Bcolor+String	set dot matrix, set string
Point set	0x50	(x,y)0+(x,y)1+……+(x,y)n	set several points of backgroud color(removed points)
	0x51		set several points of foregroud color
	0x74	X+Ys+Ye+Bcolor+$(y,Fcolor)_0$+……+$(y,Fcolor)_n$	fast points setting of dynamic curve
	0x72	Address(H:M:L)+$Data_word_0$+……+$Data_word_n$	direct screen access
Segments and polygon	0x56	(x,y)0+(x,y)1+……+(x,y)n	connect set points using foreground color(display polygon)
	0x6D		connect set points using background color(remove polygon)
	0x75	X+Y+Height_max+$Height_0$+$Height_1$+……+$Height_n$	fast display continuous vertical segments with the same end(spectrum)
	0x76	X+X_dis (0x00-0xFF) +Y_0+Y_1+……+Y_n	fast display fold lines
Arc and circular domain	0x57	(Type,x,y,r)0+(Type,x,y,r)1+……+(Type,x,y,r)n	invert color/display several arcs and circular domains
Rectangular box	0x59	(xs,ys,xe,ye)0+(xs,ys,xe,ye)1+……+(xs,ys,xe,ye)n	display several rectangular boxes in foreground(display rectangular boxes)
	0x69		display several rectangular boxes in background(remove rectangular boxes)

Area operations	0x52	None	clear screen
	0x5A	(xs,ys,xe,ye)0+(xs,ys,xe,ye)1+……+(xs,ys,xe,ye)n	clear several set areas
	0x5B		fill several set areas
	0x5C		invert color of several set areas
	0x60	(xs,ys,xe,ye,n)0+(xs,ys,xe,ye,n)1+……+(xs,ys,xe,ye,n)n	left-circular shift of several set areas
	0x61		right-circular shift of several set areas
	0x62		move left of several set areas
	0x63		move right of several set areas
Image/Icon display	0x70	Picture_ID	display a saved full-screen image in terminal
	0x71	Picture_ID+X_s+Y_s+X_e+Y_e+X+Y	display a cutted part of a saved image in terminal
	0xE2	Picture_ID	save the current-screen image into terminal
	0x99	$(x,y,Icon_ID)_0$+……+$(x,y,Icon_ID)_n$/none	user defined icon display
Animation support	0x9A	ID_0+ID_1+……+ID_n	open/close user pre-defined icon
Temporary buffer	0xC0	Address(H:L)+$Data_word_0$+……+$Data_word_n$	write data into temporary buffer
	0xC1	0x01+Address+Pixel_Number(H:L)	display preset data points of temporary buffer
		0x02+Address+Line_Number(H:L)	display preset data lines of temporary buffer
		0x03+Address+X+Y+Line_Number+D_x+Dis_x+K_y+Color	connect data points to lines in temporary buffer(dynamically scale curve)
		0x04+Adr1+X+Y+Line_Number+0x01+Dis_x+Color1+Adr0+Color0	fast-flickerless connect data points to lines in temporary buffer(oscilloscope)
Database operations	0xF2	0xF2+0xF2+0x5A+0xA5+Lib_ID	modify the character library room
	0x90	0x55+0xAA+0x5A+0xA5+Address(H:MH:ML:L)+Data	write data into user database(32MB)
	0x91	Address+Read_Length(H:L)	read data from user database(32MB)
Keyboard operations	0x71	K_code	upload key code
	0xE5	0x55+0xAA+0x5A+0xA5+K_Code0+……+K_code63	configure key code interface
Touch panel operations	0x72	Touch_X+Touch_Y	upload data the last time after release the touch panel(use 0xE0 to set closed)
	0x73		upload data when press the touch panel(use 0xE0 to set upload only once)
	0xE4	0x55+0xAA+0x5A+0xA5	calibrate the touch panel
	0x78	Touch_Code	automatically upload pre-defined key code when in the mode of touch panel automatic switch over

Buzzer control	0x79	None	buzzer moo once
Backlight control	0x5E	None/0x55+0xAA+0x5A+0xA5+V_ON+V_OFF+ON_TIME	close backlight or set touchpad(keyed) backlight mode
	0x5F	None/PWM_T (0x00-0x3F)	open backlight or adjust backlight brightness in PWM
Clock operation	0x9B	0x5A、0x5B(read)/0x00(close)/0XFF+M+TM+Color+X+Y(open)	open/close automatic overlapping clock display
	0xE7	0x55+0xAA+0x5A+0xA5+YY:MM:DD:HH:MM:SS	set clock
Parameters configuration	0xE0	Bode_Set+Para1	configure user serial rate, upload format of touch panel, backlight control mode
Configuration files operation(simplified OS)	Pic_Now+(xs,ys,xe,ye)+P_next+P_cut+Touch_Code		automatically switch of touchpad interface(0x1E character library file)
	Pic_Now+0x00:K_Code+Pnext+(Pcut,xs,ys,xe,ye,		automatically switch of keyed interface(0x1B character library file)
	0x5A+Mode+Pic_ID+(xs,ys,xe,ye)+(xt,yt)		automatically animation play(0x1C character library
	Pic_ID+(xs,ys,xe,ye)		define icon character(0x1D character library file)
	Command_Length+Command_String		user pre-defined touchpad upload commands(0x1A character library file)
			user commands configuration files(0x1F character library file)

3. Command Description

3.1.1 Power on (0x00 0A)

TX: None

RX:AA 00 0A CC 33 C3 3C

Attention: 1. The command is needed to be set by USB mini port . " AA 00 0A CC 33 C3 3C"

2. The display terminal will automatically send the 00 0A command after power on, to inform user's MCU.

3.1 Handshake(0x00)

Tx: AA 00 CC 33 C3 3C

Rx: AA 00 'OK_V*.*' *P1 P2 P3* Pic_ID CC 33 C3 3C

For example: AA 00 4F 4B 5F 56 04 2E 06 03 07 2A 00 00 CC 33 C3 3C (5.6" terminal with V4.6 version)

- OK_V*.*， *.* is the current version of display module
- P1 is the resolution of the screen.

P1	0x00	0x01	0x02	0x03	0x04	0x05	0x06
resolution	320×240	480×272	640×480	800×480	800×600	240×320	272×480
P1	0x07	0x08	0x09	0x0A	0x0B	0x0C	0x0D
resolution	480×640	480×800	600×800	1024×768	768×1024	1024×600	600×1024

- P2 is the serial baud rate set by current users, the parameters are set by USB mini port.

P2	0x00	0x01	0x02	0x03	0x04	0x05	0x06	0x07
Baud rate	1200	2400	4800	9600	19200	38400	57600	115200

- P3 is the configuration value of the touch screen keys, the values are set by USB mini port.

Para1	Bit description

Para1[7]	0 = when release touch screen or keyboard, automatically upload 0x72 command; the touch control model must be set as 0. When release keyboard, automatically upload 0x72 command; the keyboard control model must be set as 0. 1 = when release touch screen, not upload 0x72 command. when release keyboard, not upload 0x70 command.
Para1[6]	Under coordinate or keyboard return back mode 0 = when press touch screen, automatically upload 0x73 command every 100mS until releasing touch screen. When press keyboard, automatically upload 0x71 command every 200mS until releasing keyboard. 1 = upload 0x73 command automatically for once only when pressing touch screen. upload 0x71 command automatically for once only when pressing keyboard. Under touch control mode and Para[0]=1: 0 =when press touch screen, upload 0x79 command automatically every 100mS until releasing touch screen. 1 = upload 0x79 command automatically for once only when pressing touch screen. Under keyboard control mode and Para[0]=1: 0 = when press keyboard, automatically upload 0x79 command every 200mS until releasing keyboard. 1 = upload 0x79 automatically for once only when pressing keyboard.
Para1[5]	0 = forbid touch control or keyboard control function 1= activate touch control and keyboard control function, the display terminal automatically switch the user interfaces according to the touch control and the keyboard control configuration files.
Para1[4]	0 = touch screen and keyboard work normally 1 =forbid touch screen or keyboard to work
Para1[3]	0 =activate touch screen and keyboard with buzzer when pressing 1 =forbid touch screen and keyboard with buzzer when pressing
Para1[2]	Undefined.
Para1[1]	0 = under touch control mode, the buzzer is always on; 1= under touch control model, the buzzer rings for once only when pressing the valid area; when Para[1]=1, the Para[3]=1
Para1[0]	Under touch control mode and keyboard control mode: 0 = not upload 0x79 command 1 = upload 0x79 command

- Pic_ID is the sequence number of the current picture.

Display module needs 0.5～2 seconds for initialization and will no respond to any command until the initialization is finished. So users can confirm whether the initialization process is finished by sending power on 0x000a command or handshake command.

3.2 Set the current color palette (0x40)

TX: AA 40 <FC> <BC> CC 33 C3 3C

Rx: None

- <FC>: *foreground color palette, 2 bytes(16 bit, 65k color), default reset value is 0xFFFF(white)*
- <BC>: *background color palette, 2 bytes(16 bit, 65k color), default reset value is 0x001F(blue)*
- color palette of 16 bit is defined to 5R6G5B mode as table below:

16 bit definitions of Color Palette																
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Define	R4	R3	R2	R1	R0	G5	G4	G3	G2	G1	G0	B4	B3	B2	B1	B0
	Red 0xF800					Green 0x07E0						Blue 0x001F				



Once set, it will maintain that way unless it is reset; it will restore the default values once display module hardware resets.

3.3 Set character spacing(0x41)

TX: AA 41 <D_X> <D_Y> CC 33 C3 3C

Rx: none

- <D_X>: *Character spacing in X direction (column spacing), ranging from 0x00 to 0x7F. Default reset value is 0x00.*
- <D_Y>: *Character spacing in Y direction (line spacing), ranging from 0x00 to 0x7F. Default reset value is 0x00.*



Once set, it will maintain that way unless it is reset; it will restore the default values once HMI hardware resets.

3.4 Pick specified position's color(0x42，0x43)

Tx: AA <CMD> <X> <Y> CC 33 C3 3C

Rx: None

- <CMD>: command 0x42 accesses specified position's color to background color palette; command 0x43 accesses specified position's color to foreground color palette.
- <X> <Y>: specified position's coordinate.(the coordinate is represented with 2 bytes)
  e.g.
  AA 42 00 10 01 00 CC 33 C3 3C
  to access the color of position x=16（0x0010） y=256（0x0100）to background color palette.

3.5 Cursor display (0x44)

Tx: AA 44 <Cursor _EN> <X> <Y> <Cursor _Width> <Cursor _Height> CC 33 C3 3C

Rx: None

- <Cursor _EN>
  0x01 Turn on cursor display on and the cursor will display at (x ,y)
  0x00 Turn off cursor display
- <X>, <Y> is the coordinate of the top left corner of the cursor
- <Cursor _Width> the width of cursor, ranging from 0x01 to 0x1F
- <Cursor _Height> the height of cursor, ranging from 0x01 to 0x1F



If cursor display is turned off, other parameters in this command are of no use.

e.g.

AA 44 01 00 80 00 60 10 03 CC 33 C3 3C

to display a cursor of 16*3 dot matrix at (128,96)

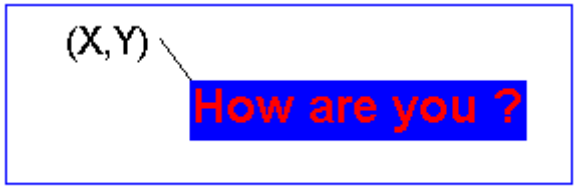
3.6 Text display

3.6.1 standard character library display（0x54，0x55，0x6E，0x6F，0x98，0x45）

Tx: 54 <X> <Y> <String> CC 33 C3 3C

Rx: None

- 0x54: display 8*16 dot matrix ASCII character
- 0x55: display 16*32 dot matrix ASCII character
- 0x6E: display 6*12 dot matrix ASCII character
- 0x6F: display 12*24 dot matrix ASCII character
- <X> <Y> Coordinates of the left top corner of the first character
- <String> string data is 1 byte Unicode. Colors are set by command 0x40; Character interval is set by command 0x41; and auto-wrap to next line at the end of line. Command 0x0D & 0x0A represent "back to line start" and "wrap to next line"



e.g.

AA 55 00 80 00 30 48 6F 77 20 61 72 65 20 79 6F 75 20 3F CC 33 C3 3C

to display string "How are you?" at (128,48)

3.6.2 Character Library Display（0x98）

Tx: AA 98 <X> <Y> <Lib_ID> <C_mode> <C_dots> <F color> <B color> <String> CC 33 C3 3C

Rx: None

- <X> <Y> Coordinate of the left top corner of the first character
- <Lib_ID> Character Library ID, ranging from 0x00 to 0x0F.

The definition of the font library is as follows:

Lib-ID	Capacity	Description
0x00-0x07	256K	The small Custom character library
0x08-0x0F	1M	The big Custom character library
0x20		6x12 dot matrix ASCII character
0x21		8x16 dot matrix ASCII character
0x22		12x24 dot matrix ASCII character
0x23		16x32 dot matrix ASCII character
0x24		24x48 dot matrix ASCII character
0x25		32x64 dot matrix ASCII character
0x26		48x96 dot matrix ASCII character
0x27		64x128 dot matrix ASCII character

Bit	.7-.4	.3-.0
Definition	Display Mode	Character Encoding Method

Description	.7=1 foreground color display =0 foreground color not display .6=1 background color display =0 background color not display .5=1 longitudinal display =0 transverse display .4=1 Recover background =0 Not recover background	0x00 8bit ASCII Character library contains most 256 bytes 0x01 GB2312 Chinese internal code 0x02 GBK Chinese extended internal code or Korea HANGUL code 0x03 BIG5 Traditional Chinese Code 0x04 SJIS Japanese Code **0x05 UNICODE UNICODE Code（UTF16）** 0x06-0x0F: Undefined

➢ <C _Mode> to select Text Display Mode and Encoding Mode as follows:

For example:

If to display both foreground and background colour, the C_Mode is 1100 0005 (Hex 0xC5)

If to display foreground and not to display background colour, the C_Mode is 1000 0005 (Hex 0x85)

➢ <C _dots> character size setting

C _Dots	Lib_ID	
	0x00 – 0x07	0x20 – 0x27
0x01	6*12	
0x02	8*16	
0x03	12*24	
0x04	16*32	
0x05	20*40	
0x06	24*48	
0x07	28*58	
0x08	32*64	
0x09	12*12	
0x0A	16*16	
0x0B	24*24	
0x0C	32*32	
0x0D	40*40	
0x0E	48*48	
0x0F	56*56	
0x10	64*64	
0x11	40*80	
0x12	48*96	
0x13	56*112	
0x14	64*128	
0x15	80*80	
0x16	96*96	
0x17	112*112	
0x18	128*128	
0x19	6*8	
0x1A	8*10	
0x1B	8*12	
0x1C	100*200	
0x1D	200*200	
0x1E	64*48	

- <Fcolor> foreground color of character
- <Bcolor> background color of character
- <String> string data is 2 byte Unicode, display spacing is set by command 0x41 and automatically wrap at the end of a line.

e.g.: AA 98 00 80 00 30 00 C5 04 F8 00 00 1F 00 31 00 32 00 33 00 41 00 42 00 43 CC 33 C3 3C
to display 16*32 dot size string "123ABC" at (128,48) position.

3.6.3 Set Text box（0x45）

1. Set Text box
   Tx: AA 45 <Xs> <Xs> <Xe> <Xe> CC 33 C3 3C
   Rx: None
- < Xs > <Xs> <Xe> <Xe> is the Coordinate of Text box.

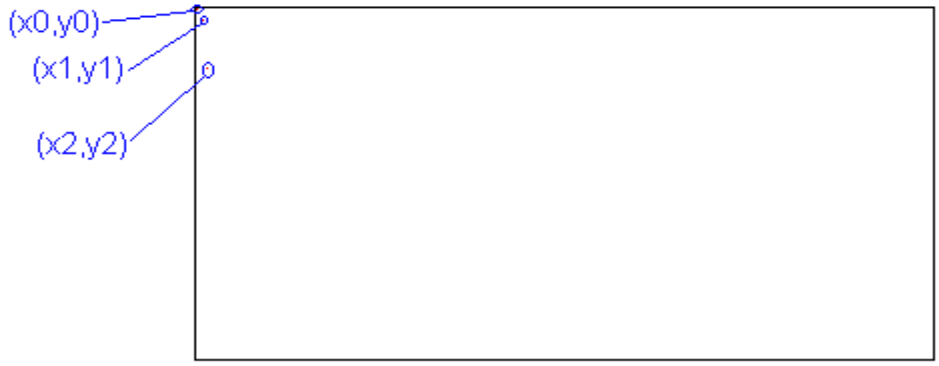
After set the text box, the text will automatically wrap in the limited area.
2. Cancel Text box
   Tx: AA 45 00 CC 33 C3 3C
   Rx: None

3.7 Point display(0x50，0x51，0x74，0x72)

3.7.1 set point（0x50，0x51）

Tx: AA <CMD> <(x0, y0) (x1,y1) ……(xi, yi)> CC 33 C3 3C
Rx: None
- <CMD>
  0x50: display point using background color(delete point)
  0x51: display point using foreground color(set point)
- <(x0,y0) (x1,y1) ……(xi,yi)> coordinates to be set; and the maximum point number of one frame is 62.

e.g.
AA 51 00 00 00 00 00 03 00 06 00 05 00 20 CC 33 C3 3C
to display 3 points using foreground color at (0,0),(3,6),(5,32) as follows:

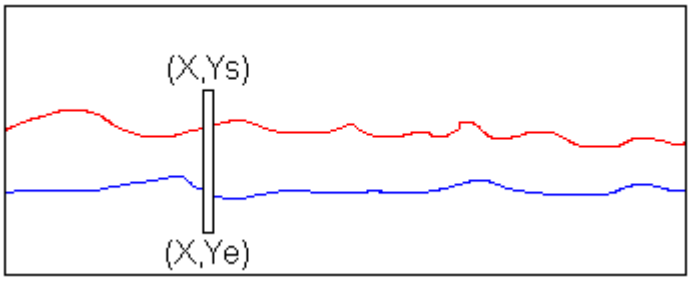


3.7.2 Dynamic Curve Display (0x74)

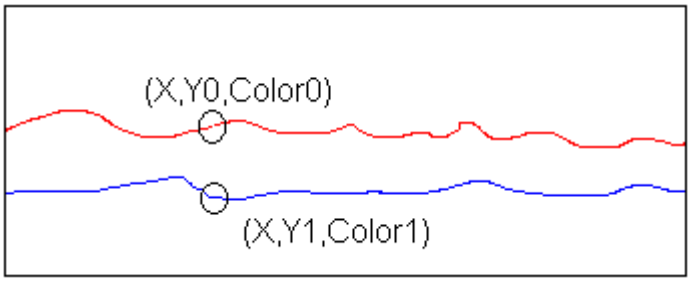
Tx: AA 74 <X> <Ys> <Ye> <Bcolor> <(Y0,Fcolor0), (Y1,Fcolor1)……(Yi, Fcolor i)> CC 33 C3 3C
Rx: none

This command is mainly used for implementation of multi-dynamic curves on single screen. Terminals process commands by following steps:

Step 1: Erase the vertical line from (X,Ys) to (X ,Ye), using specified color<Bcolor>. Empty the original content.



Step 2：Set point at (X,Yi), using specified color<Fcolori>.



This command will not alter Palette's default settings.

3.7.3 Direct Display Operation (0x72)

Tx: AA 72 <Address _H:M:L> <Pixel_data0……Pixel _data n> CC 33 C3 3C
Rx: None

This command is mainly used to download figures to Terminal, which users generally do not use.

USB write video memory directly: USB adopt batch transmission, the data length is fixed 64 bytes

1.AA 72 AA 55 A5 5A <Xs><Ys><Width><Height>CC 33 C3 3C…(64 bytes)
2.Distribute picture's data

3.8 Line Display(0x56, 0x5D，0x75，0x76)

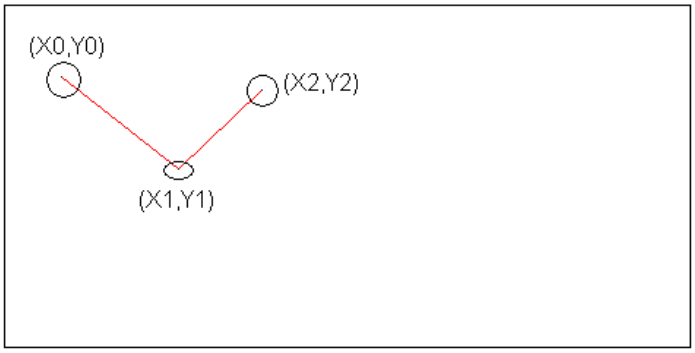
3.8.1 Connecting specified points（0x56，0x5D）

Tx: AA <CMD> <(x0,y0) (x1,y1) ……(xi, yi)> CC 33 C3 3C
Rx: None

- <CMD>
  0x56: Connect specified points by lines, using foreground color(set by command 0x40).
  0x5D: Connect specified points by lines, using background color(set by command 0x40).
- <(x0,y0) (x1,y1) ……(xi,yi)>: coordinates of specified points.

e.g.：

AA 56 00 28 00 32 00 78 00 70 00 B1 00 3A CC 33 C3 3C
Connect 3 points, (40,50),(120,112),(177,58), to lines, using foreground color, as follows:



3.8.2 Connecting points with specified color (0x6A01)

Tx：AA 6A 01<Cur_Clr> <(x0，y0) (x1，y1)……(xi，yi)> CC 33 C3 3C
Rx：none

- <Cur_Clr >2 bytes parameters, indicates the color used for connecting line;
- (x0，y0) (x1，y1)……(xi，yi)> are coordinates of connecting points, all should be 2 bytes parameters.

3.8.3 Display circle with specified color (0x6a02)

Tx：AA 6A 02 <Cur_Clr><(X_0,Y_0,R_0) ……(X_i,Y_i,R_i)>CC 33 C3 3C

Rx：NONE

□ □ <Cur_Clr>2 bytes parameter, color of the line；

□ □ <X_i> <Y_i>coordinates of the center of the circle；

□ □ <R_i>radius of the circle, 1 byte parameter, ranging from 0x01-0xFF；

3.8.4 Fill the circular domain with specified color(0x6a03)

Tx：AA 6A 03 <Cur_Clr><(X_0,Y_0,R_0) ……(X_i,Y_i,R_i)>CC 33 C3 3C

Rx：无

□ □ <Cur_Clr>2 bytes parameter, color that fill the circular domain;

□ □ <X_i> <Y_i>coordinates of the center of the circle in the circular domain；

<R_i> radius of the circle, 1 byte parameter, ranging from 0x01-0xFF；

3.8.5 Ellipse display（0x58）

Tx：AA 58（M1+X1+Y1+Rx1+Ry1）+…….+(Mn+Xn+Yn+Rxn+Ryn) CC 33 C3 3C

Rx：None

X，Y，2 bytes parameter，coordinates of the center of the ellipse.

Rxi，one byte parameter, horizontal radius of the ellipse.

Ryi，one byte parameter, vertical radius of the ellipse.

Mi，one byte parameter, define the ellipse drawing mode.

M＝0 invert color ellipse； M＝1 draw a ellipse； M＝2 invert color ellipse domain；

M＝3 draw ellipse domain； M＝4 delete the ellipse； M＝5 delete the ellipse domain

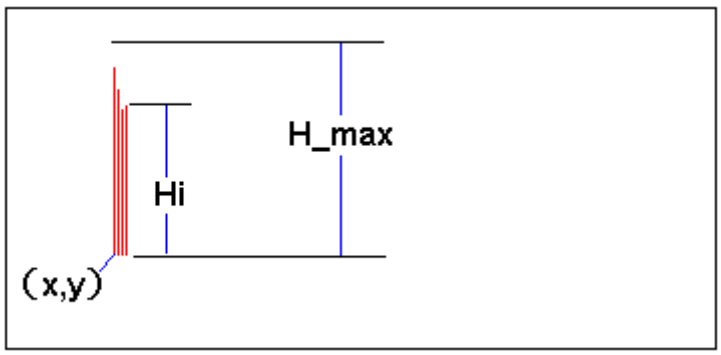
This command uses the color set by 0x40 command.

3.8.6 Spectrum Display（0x75）

Tx: AA 75 <(x，y)>，<H_max>，<H0……Hi> CC 33 C3 3C

Rx: None

- <(x ,y)>: x is start coordinate on spectrum's X-axis. x=x+1 after displaying another spectrum line; y is horizontal position on spectrum. Starting and end coordinates of every spectrum line's are y and (y-Hi) respectively.
- <H _max> Maxim height of a spectrum line.
  if H_max=0x01-0xFF, spectrum line's height Hi is a variable of 1 byte.
  if H_max=0x00, the following 2 bytes represent Hmax and Hi is a variable of 2 bytes.
- <H0……Hi>: height of one spectrum line, one or two bytes.
  Spectrum line's color is set by Palette Command 0x40 .When display, spectrum line, height Hi, will display by foreground color, other spectrum lines, height H_max-Hi, will display(be eresed) by background color.



3.8.7 Broken Line Display（0x76）

Tx: AA 76 <x>，<x _dis>，<Y0……Yi> CC 33 C3 3C
Rx: None

- <x>: the start coordinate on broken line's X-axis. x=x+x_dis after connecting a point by line.
- <x_dis>: the increment of coordinate on X-axis.
- <Y0……Yi>: the top points' coordinates of broken lines, using foreground color to display.

This command's function is similar to command 0x56 except that coordinate X here is auto-calculated by HMI for fast connecting.

3.8.8 Connecting points according to offset values (0x78)

Tx：AA 78 <x，y> <dx0，dy0> <dx1，dy1>……<dxn，dyn> CC 33 C3 3C
Rx：none

- <x,y> indicates the starting point coordinate, 2 bytes parameters
- <dxn,dxy>indicates 1 byte offset value in both X and Y axises. The highest digit(.7) is symbol bit,"1" represents the negative,"0" represents the positive .

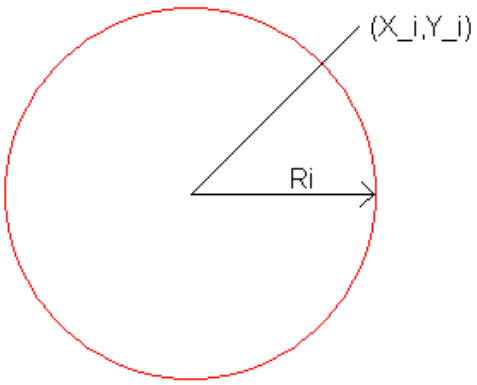
3.9 Arc Curve display(0x57)

3.9.1Arc and Circular domain display（0x57）

Tx: AA 57 (<Type_0> <X_0> <Y_0> <R_0>) ……(<Type_i> <X_i> <Y_i> <R_i>) CC 33 C3 3C
Rx: None

- <Type_i> to select type
  0x00 display specified arc, using invert color
  0x01 display specified arc, using foreground color (set by command 0x40)
  0x02 display specified circular domain, using invert color
  0x03 fill specified circular domain, using foreground color (set by command 0x40)
- <X_i> <Y_i> coordinates of the center of arc or circular domain
- <R_i> radius of the arc or circular domain, ranging from 0x01 to 0xFF

e.g.:
AA 57 01 00 60 00 60 40 CC 33 C3 3C
to display an arc, using foreground color, with the center (96,96) and radius 64 as follows:



3.9.2 Arc segment display（0x5704）

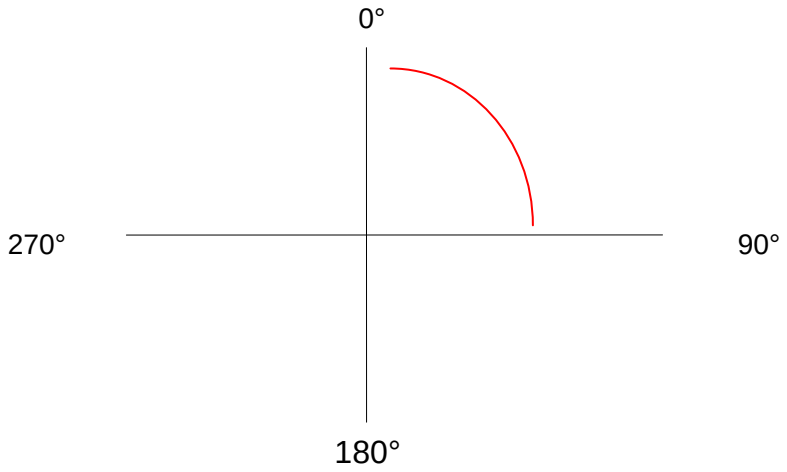
Tx: AA 57 04〈X〉〈Y〉〈R〉〈A_S〉〈A_E〉CC 33 C3 3C
Rx: None

- 〈X〉〈Y〉coordinates of the center of arc
- 〈R〉 radius of the arc,0x0001-0x03FF
- 〈A_S〉start angle of arc,0x00-0x02D0(0-720),units 0.5°
- 〈A_E〉end angle of arc，0x00-0x02D0(0-720),units 0.5°

Display using the foreground set by 0x40 command.

e.g.:
AA 57 04 00 60 00 60 00 40 00 00 00 B4 CC 33 C3 3C

to display an arc, using foreground color, with the center (96,96) and radius 64 ,start angle 0°， end angle 90°as follows:



3.10 Area Display

3.10.1Rectangle box or rectangle domain display(0x59，0x69，0x5A，0x5B，0x5C，0x65)

Tx: AA <CMD> (<Xs_0> <Ys_0> <Xe_0><Ye_0>) ……(<Xs_i> <Ys_i> <Xe_i><Ye_i>) CC 33 C3 3C
Rx: None

- <CMD >
  0x59: to display rectangle box, using foreground color (set by command 0x40), with one dot matrix line width
  0x69: to display rectangle box, using background color (set by command 0x40), with one dot matrix line width
  0x5A: to fill rectangle domain, using background color (set by command 0x40)
  0x5B: to fill rectangle domain, using foreground color (set by command 0x40)
  0x5C: to display specified rectangle domain, using invert color (set by command XOR 0xFF). Reverted when using invert color again.
  0x65 to cover specified rectangle area with foreground color.
- <Xs_i> <Ys_i> <Xe_i> <Ye_i>: (Xs_i,Ys_i) is top left coordinate of rectangle box or domain; and (Xe_i,Ye_i) is bottom right coordinate. All command parameters shall be 2 bytes.

e.g.:

AA 5C 00 40 00 40 00 80 00 80 CC 33 C3 3C

to invert color of rectangle domain specified by top left coordinates (64,64) and bottom right coordinate (128,128) as follows:



3.10.2 Area Fill (0x64)

Tx: AA 64 <x> <y> <color> CC 33 C3 3C
Rx: None

- <x> <y> the position of the seed point to area fill

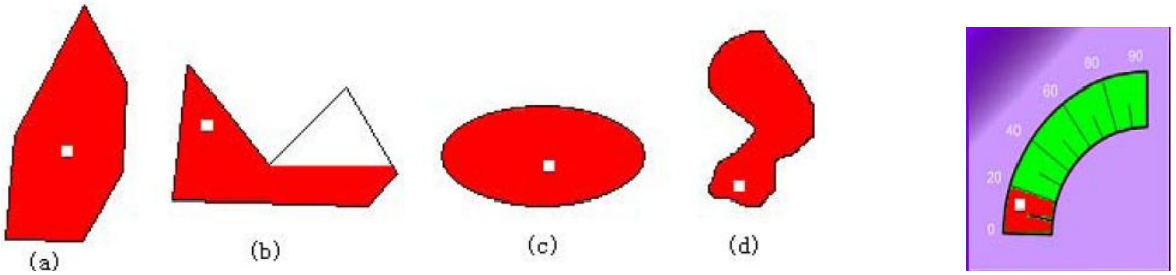
- <Color>: color to use

Notice:

a. The initial color of the filling area shall be the same with that of seed point , otherwise , Only the area having the same color with the seed point will be filled.(Area with the other color is handled as the border)

b. It fits for filling the convex polygon area. To the concave polygon, some area can not be filled (as shown in figure b). However, you could set more seed points of different positions to realize full filling.

c. *This command will not alter Palette's default settings.*



The effect of area filling (the white is the position of the seed point and it is filled actually)

e.g.

AA 64 00 64 00 64 F8 00 CC 33 C3 3C

3.11 Clear Screen（0x52）

Tx: AA 52 CC 33 C3 3C

Rx: None

to fill full screen with background color(set by command 0x40)(clear screen)

3.12 Area Selected Movement(0x60，0x61，0x62，0x63)

Tx: AA <CMD> (<Xs_0> <Ys_0> <Xe_0><Ye_0> <N_0>) ……(<Xs_i> <Ys_i> <Xe_i><Ye_i> <N_i>) CC 33 C3 3C

Rx: None

- <CMD >

0x60: selected area for right-circular shift , i.e to move the far-left area shift to far-right.

0x61: selected area left-circular shift,i.e.,to move the far-right area shift to far-left.

0x62: to move selected area right ,i.e to move from right to left,far-left area lost, using background color to fill far-right area(set by command 0x40)

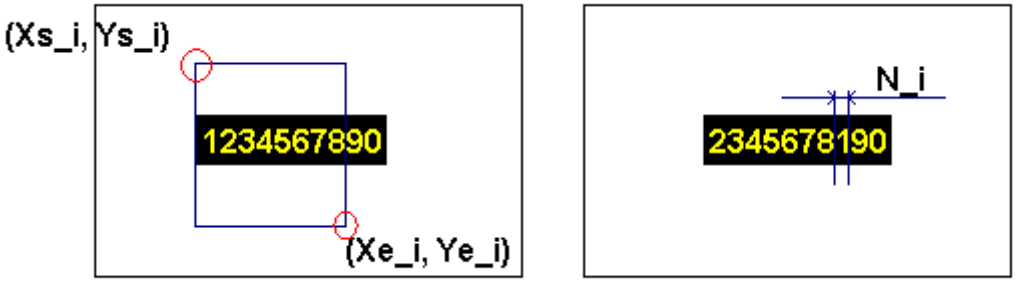
0x63: to move selected area left,i.e.to move from left to right,far-right area lost, using background color to fill far-left area(set by command 0x40)

- <Xs_i> <Ys_i> <Xe_i><Ye_i>: (Xs_i,Ys_i): top left and bottom right coordinates of selected area.
- <N_i>: numbers of dot matrix in area movement, ranging from 0x01 to 0x0F.

e.g.:

AA 60 00 40 00 40 00 80 00 80 08 CC 33 C3 3C

to move the selected area at top left coordinate(64,64) and bottom right coordinate(128,128) for 8 dot matrix from right to left as follows:



Note: this command can not be used in case the version supports text 90 degree rotation.

3.13 Image or Icon Display（0x70，0x71，0x99，0xE2, 0x7B）

3.13.1 Image Display（0x70）

The command display full width image, the resolution of image and resolution of the display terminal shall be the same.

Tx: AA 70 <Pic_ID> CC 33 C3 3C

Rx: None

<Pic_ID> indicates the images' index ID that are saved in the flash memory of SDW，2 bytes parameters.

For example:

AA 70 01 02 CC 33 C3 3C display the 258th image in the SDW

Note: After executing the command, the images' index ID in system will be automatically modified.

3.13.2 Arbitrary small format image display (0x70 01)

The 0x70 command of image display in full screen requires the resolution of images and the resolution of the terminal be the same. In order to display small format images, the small format images are usually connected together to a full format image whose resolution is the same as the terminal's, and then use 0x71 cut command or 0x99 custom icon command to display (as shown below). On the occasion where a large number of small images are to be displayed, the above method seems a bit complicated.

The 0x70 01 command of arbitrary small format image display is pretty convenient to display the small images, while the command requires the resolution of images shall be less than or equal to the actual resolution of the terminal.

Tx: AA 70 01 < L_Pic_ID > <X><Y>CC 33 3C C3

- <L_Pic_ID> indicates arbitrary images' index ID that are saved in SDW series Flash memory, 2 bytes parameters;
- The <X> <Y> (X, Y) the left upper corner coordinate when a small format image is displayed in the current screen, 2 bytes of parameters;

Note: after executing the command, the images' index ID in system are not modified.

Before using the small picture command, a library file of arbitrary images needs to be created and downloaded into the terminal. The steps are as follows:

1 all the arbitrary small format images files should be saved in a folder;

2 open the arbitrary picture generator by 'development assistant - > special command- >download the file of arbitrary images - >create the file" as shown below:

3 click the red button in upper right, select the folder, upload all the arbitrary images;

4. after selecting a picture,the L_Pic_ID can be modified by move up / down button on the right.

5 click the "create a new file" in the lower left corner, to create an arbitrary picture library file, and exit.

6. Via "development assistant - > special command - > download the file of arbitrary images - > download files", and then the arbitrary picture file created is downloaded into the display terminal.

3.13.3 Arbitrary large format images displayed by cutting(0x70 02)

The command is mainly used in displaying large format image by cutting, for example, as far as a map navigation application is concerned, the map format is usually greater than the actual resolution of the terminal, and the different areas are displayed by moving the image on screen.

Like displaying arbitrarily small format images, before using the arbitrary large format images displayed by cutting command, firstly create a library file of arbitrary format images by arbitrary picture generator and download it into the terminal. The specific steps please refer to 0x70 01 command.

Arbitrary small format pictures and arbitrary large format pictures are mixed together to create an library file of arbitrary format images.

Tx：AA 70 02 <L_Pic_ID> <Xs> <Ys> <Xe> <Ye> <X> <Y>CC 33 C3 3C

Rx：None

- <L_Pic_ID>indicates arbitrary images' index ID that are saved in SDW series Flash memory,2 bytes of parameters;
- <Xs> <Ys> <Xe> <Ye> Both 2 byte parameters.

(Xs,Ys) are the upper left corner coordinates of the cutting area in the original image, (Xe，Ye) are the bottom right corner coordinates.

- <X> <Y> (X，Y) are the upper left corner coordinates of the cut image in the current position of screen, 2 byte parameters.

3.13.4 Display one image and figure up (0x7B)

Tx:AA 7B <Pic_ID> CC 33 C3 3C

Rx:AA 7B <CheckSum_H:L> CC 33 C3 3C

<Pic_ID>: index ID (set by command 0xE2) of the image saved in HMI Flash Memory

<CheckSum_H:L>: 2 Byte sum of the content of the present image

This command is used to verify the image downloaded to the flash, to ensure that the download is correct.

3.13.5 Cut Icon(0x71、0x9C、0x9D)

Tx: AA 71 <Pic_ID> <Xs> <Ys> <Xe> <Ye> <X> <Y> CC 33 C3 3C
Or AA 9C <Pic_ID> <Xs> <Ys> <Xe> <Ye> <X> <Y> CC 33 C3 3C
Or AA 9D <Pic_ID> <Xs> <Ys> <Xe> <Ye> <X> <Y> CC 33 C3 3C

Rx: None

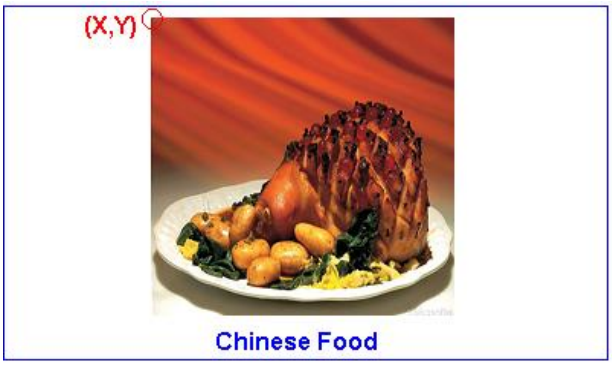
- <Pic_ID>: index ID(set by command 0xE2) of icons saved in HMI Flash Memory
- <Xs> <Ys> <Xe> <Ye>: (Xs, Ys): the top left coordinate of cutting area in original icon; (Xe,Ye): the bottom right coordinate of cutting area.
- <X> <Y>: (X,Y): the top left coordinate of cut icon in current screen

e.g.:

AA 71 00 08 01 90 00 00 03 1F 01 90 00 C8 00 14 CC 33 C3 3C

to cut 8th icon's area selected by (400,0) and (799,400) and display at(200,200) in current screen as follows:





The difference between 0x9C command and 0x71 command is that, the background color of the cutting part is transparent as using 0x9C and 0x9D (Background color must be single color and be same with the system background color), and the original background is recovered again by using 0x9C in next time, the original background is not recovered again by using 0x9D in next time.

3.13.6 User-defined Icon Display（0x99）

Tx: AA 99 <X0,Y0,Icon_ID0>……<Xn,Yn,Icon_IDn> CC 33 C3 3C
Rx: None

- <Xn,Yn>: the display position of user icon
- <Icon_IDn>: index ID of user icon, 2 bytes

See Content 3.35 below.

3.13.7 Save current screen image in HMI（0xE2）

Tx: AA E2 <Pic_ID> CC 33 C3 3C
Rx: AA E2 4F 4B CC 33 C3 3C

- <Pic_ID>: image index ID to be saved，2 byte parameter.

3.13.8 Refresh the full screen for once compulsorily (0xD0)

Tx：AA D0 CC 33 C3 3C
Rx：None

The display terminal automatically refresh images in full screen according to the images‘ index IDs that are saved in system.

3.14 Backlight Brightness Control（0x5E, 0x5F）

3.14.1 Turn off backlight（0x5E）

Tx: AA 5E CC 33 C3 3C
Rx: None

3.14.2 Commands start screen saver (0x5E)

Tx：AA 5E aa 55 a5 5a CC 33 C3 3C
Rx：None
Description:
(1) Before using the screen saver function, you need to activate screen saver function through the toolbox firstly, and set the start and close brightness of the screen saver as well as the delay time;
(2) If the touch screen receives no operation during the set delay time, the display terminal will automatically enter the screen saver mode; when the touch screen receive new operation, the display terminal will automatically resume to working mode;
(3) When user’s MCU send AA 5E aa 55 a5 5a CC 33 C3 3C to the display terminal, the terminal will enter the screen saver mode immediately; when there is a new operation in the touch screen, the display terminal will automatically resume to working mode.

3.14.3 turn the backlight to maxim brightness（0x5F）

Tx: AA 5F CC 33 C3 3C
Rx: None

3.14.4adjust backlight brightness（0x5F）

Tx: AA 5F <PWM_T> CC 33 C3 3C
Rx: None

- <PWM_T>: backlight brightness value controlled by PWM, ranging from 0x00 to 0x64; 0x00 means no backlight while 0x64 represents to maxim brightness.

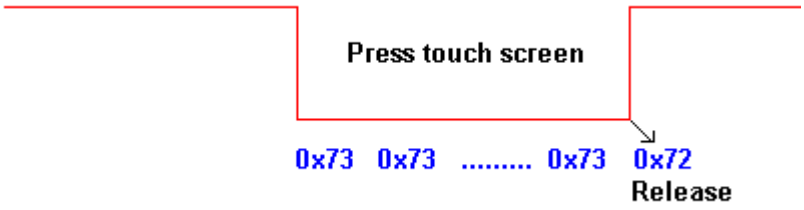
For backlight mode CCFl in HMI, backlight brightness cannot be controlled by PWN; it is open/off way.

3.15 Touch Panel Operations (0x72，0x73，0x78，0x79，0xE4)

3.15.1 touched position automatically upload(0x72，0x73)

When click the touch panel, HMI can automatically upload coordinate of touched position as following format
Tx: None
Rx: AA 73 <X> <Y> CC 33 C3 3C to upload the coordinate of touch position when the user press the touch panel, one or more times(set by command 0xE0)
AA 72 <X> <Y> CC 33 C3 3C to upload the coordinate touch position when the user release the touch panel, only once(set by command 0xE0)

- <X> <Y>: coordinates of touched position, corresponding to screen resolution



If user uses the touchpad function (set by command 0xE0) and enables the function that uploads nothing when clicking invalid area, then clicking the touch panel will not leading to uploading command 0x72 and 0x73.

3.15.2 Automatically upload key code(0x78，0x79)

if user enable touch pad or keyed function(set by command 0xE0) and key code back transmission, then when clicking the valid area or keys, HMI will upload user pre-defined key code of 2 bytes.

Tx: None

Rx: AA 79 <Touch_Code> CC 33 C3 3C to upload the key code when the user press the touch panel, one or more times (set by command 0xE0)

AA 78 <Touch_Code> CC 33 C3 3C to upload the key code when the user release the touch panel, only once (set by command 0xE0)

3.15.3 Read the touch batch number (0x01, only for the RS-485 interface display terminal)

Since the RS485 communication adopts half duplex transmission mode, so if using 0x78, 0x79 command to upload touch batch number in a sudden way, the communication conflict will occur easily in the display terminal. (When user sends display command, the terminal will upload touch batch number at the same time.)

In order to solve the problem of data conflicts in RS485 interface, the SDW series intelligent display terminal designed the batch number query and read command(0x01). Only when the user sends out the 0x01 command of reading touch screen batch number, will the terminal upload the touch batch number.

Tx：AA 01 CC 33 C3 3C

Rx： AA 01 <BatchNumber[0]> <BatchNumber[1]> <BatchNumber[2]> <BatchNumber[3]>CC 33 C3 3C

- BatchNumber[n] is a double bytes parameter, touch button batch number; able to record touch button batch number 4 times. When the touch button value are not read more than 4 times, then only the last 4 times records are reserved, and the latest touch button value is always saved in BatchNumber[0]. After the terminal executes the read batch number command(0x01), the BatchNumber[n] will be cleared to 0xFFFF.

Note: the command corresponds to the 0x78 command; only when it is under touch mode and the button is lifted will the touch button batch number be saved into the to BatchNumber[0].This command is also valid for keyboard control mode (using external mechanical keyboard).

3.15.4 Touch panel calibration mode（0xE4)

Tx: AA E4 55 AA 5A A5 CC 33 C3 3C

After sending commands, click three white spots showing at different positions of the screen in turn HMI will upload following commands when calibration finished

Rx: AA E4 4F 4B CC 33 C3 3C

Our unique drift compensation technique can guarantee that our products only need calibration once unless users re-equipped the touch panel

3.16 Working Mode Configuration（0xE0）

V6.2 working mode parameters are set by the USB mini port, using the configuration download software in the SDW terminal toolbox(as shown in 4-1), the working mode set command 0xE0 in the serial interface command sets keeps only four control bits, and the set parameters are no longer saved

when power off.

Tx: AA E0 <NC1><Para1><NC2>CC 33 C3 3C

Rx: AA E0 <NC1><Para1><NC2>CC 33 C3 3C

- NC1, Undefined function
- <Para1>: To set touching and keyboard work mode, described as below

Para1	Bit description
Para1[7]	Undefined
Para1[6]	Undefined
Para1[5]	0= forbid touch control and keyboard control function 1= activate touch and keyboard control function, the display terminal automatically switch the user interfaces according to the touch control and the keyboard control configuration files.
Para1[4]	0= touch screen and keyboard work normally 1= forbid touch screen or keyboard to work
Para1[3]	0 =activate touch screen and keyboard with buzzer when pressing 1 =forbid touch screen and keyboard with buzzer when pressing
Para1[2]	Undefined
Para1[1]	0 = under touch control model, the buzzer is always on; 1= under touch control model, the buzzer rings for once only when pressing the valid area; when Para[1]=1, the Para[3]=1
Para1[0]	Undefined

- NC2, Undefined function

3.17 Execute command periodically and circularly（0x9A）

The effect of this command is to save the command (such as 71 command) used periodically in the HMI beforehand, user could use the command without intervention to save code.

3.17.1 Enable the function of executing command periodically and circularly

Tx: AA 9A Pack_ID CC 33 C3 3C

Rx: None

< Pack_ID > the ID of automatic and circular command group. Each group is 8KB,and contain maximum 64 command .The space between two command is 128 byte.

3.17.2 Close the function of executing command periodically and circularly.

Tx: AA 9A FF CC 33 C3 3C

Rx: None

3.17.3 Automatic Cycle control configuration file

The automatic cycle control configuration file is the text-only file, can define at most 16 groups of commands.

Each command starts from control numerical order Pack_ID, each group can define 64 commands mostly, each command includes three parts: delay, length and content, as shown in table 8-3-1.

Table 9-6-1

Definition	Description
Command_Delay	delay time after executing the command, Unit: 8mS, 0X00 means no delay
Command_Length	length of command
Command	standard command without frame head 0XAA and frame end CC 3C C3 3C

Example of automatic cycle control configuration file

Pack_ID=0 Control0, Pack_ID=00H

60, 3, 0X70, 00, 00;(display picture0, delay 480mS)

60, 3, 0X70, 00, 01;(display picture1, delay 480mS)

60, 3, 0X70, 00, 02;(display picture2, delay 480mS)

Pack_ID=1 Control1, Pack_ID=01H

…

Pack_ID=15 Control15, Pack_ID=0FH
60, 3, 0X70, 00, 00;(display picture0, delay 480mS)
60, 3, 0X70, 00, 01;(display picture1, delay 480mS)
60, 3, 0X70, 00, 02;(display picture2, delay 480mS)
60, 3, 0X70, 00, 03;(display picture3, delay 480mS)
60, 3, 0X70, 00, 04;(display picture4, delay 480mS)

The programmed automatic cycle control configuration file can be downloaded into the display terminal through “development assistant software”.

Downloading method: development assistant software > control class > download the cycle control.

After downloading the automatic cycle control configuration file into the display terminal, you can call any group of commands to execute the cycle according to the serial number via 0x9A command. When sending the group number 0xFF, the current cycle will be stopped.

AA 9A <Pack_ID> CC 33 C3 3C

For example:

AA 9A 01 CC 33 C3 3C；480mS as the interval, display picture0 to picture 4 circularly.

AA 9A FF CC 33 C3 3C；stop the above cycle of picture0 to picture 4.

3.18 Temporary Buffer Operation（0xC0,0xC1,0XC2）

3.18.1 write temporary buffer（0xC0）

Tx: AA C0 <Address> <Word0……Wordn> CC 33 C3 3C
Rx: None

- <Address>: the starting address, word address, in temporary buffer--RAM; totally 40k word, ranging from 0x0000 to 0x9FFF.
- <Word0……Word n>: data to be written.

3.18.2 Read temporary buffer(0xC2)

- Tx: AA C2 <Address> <Data _Length> CC 33 C3 3C
- Rx: AA C2 <Data _Pack> CC 33 C3 3C
- <Address> the starting address, word address, in temporary buffer--RAM; totally 40k word, ranging from 0x0000 to 0x9FFF.
- <Data _Length> length of the data read
- <Data _Pack> data read

3.18.3 pixel setting in temporary buffer（0xC101）

Tx: AA C1 01 <Address> <Pixel _Number> CC 33 C3 3C
Rx: None

- <Address>: the starting address, word address, in temporary buffer--RAM; totally 40k word, ranging from 0x0000 to 0x9FFF.
- <Pixel_Number>: numbers of pixels with maximum 13653. Every pixel takes up 3 word unit.

pixel data format defined in temporary buffer: (X,Y,Color).

3.18.4 connecting lines using data in temporary buffer（0xC102）

Tx: AA C1 02 <Address> <Line_Number> CC 33 C3 3C
Rx: None

- <Address>: the starting address, word address, in temporary buffer--RAM; totally 40k word, ranging from 0x0000 to 0x9FFF.
- <Line_Number>: the number of lines with maximum 8191. Every line takes up 5 word unit.

line data format defined in temporary buffer: （Xs，Ys，Xe，Ye，Color).

3.18.5 display broken line graph using data in temporary buffer（0xC103）

Tx: AA C1 03 <Address> <x> <y> <Line_Number> <D_x> <Dis_x> <K_y> <Color> CC 33 C3 3C
Rx: None

- <Address>: the starting address, word address, in temporary buffer(RAM); totally 40k word, ranging from 0x0000 to 0x9FFF.
- <x>: the starting coordinate in X-axis
- <y>: the zero position(the lowest point) in Y-axis; actual position equals to y-Ly
- <Line _ Number>: the number of lines with maximum 40960, ranging from 0x000 to 0x9FFF. Every

line takes up 1 word unit.

- <D_X>: pixels' distance in read buffer, ranging from 0x01 to 0xFF, i.e. Address=Address+D_x
- <Dis_X>: the increment in X-axis, ranging from 0x01 to 0x0F, i.e. x=x+Dis_x
- <K_y>: amplification factor of Y-axis, ranging from 0x00 to 0xFF; 1/16 is the unit, i.e. K_y=32 means amplify Y-axis twice.
- <Color>: the color of lines without any variation in system palette.

the line data format in temporary buffer is defined as Ly, 2 bytes, which is the height of pixel.

3.18.6 Fast display broken line graph using data in temporary buffer（0xC104）

Tx: AA C1 04 <Adr1> <x> <y> <Line_Number> <D_x> <Dis_x> <Color1> <Addr0> <Color0> CC 33 C3 3C

Rx: None

- <x>: the starting coordinate in X-asix
- <y>: the zero position(the lowest point) in Y-axis; actual pixel position equals to y-Ly.
- <Line_Number>: number of lines with maximum 40960. Every line takes up 1 word unit.
- <D_x> fixed write 0x01
- <Dis_X>: the increment in X-axis, ranging from 0x01 to 0x0F;i.e.x=x+Dis_x after connecting.

- <Addr0> <Addr1>: the starting address (word address) in temporary buffer(RAM), totally 40k word, ranging from 0x0000 to 0x9FFF.
  <Addr0> the starting address of the history curve to erase
  <Addr1> the starting address of the curve to display
- <Color0> <Color1>: the color of lines without any variation in system palette.
  <Color0> set the color as the window's background color
  <Color1> set the color as the color of the curve to display

The line data format in temporary buffer is defined as Ly, 2 bytes, which is the height of pixel.
This command is similar to command 0xC103 except:
A. the pixels' distance in read buffer is fixed to 1.
B. before connecting pixel one, erase former pixel zero first, which realizes **flicker-free display.**
when using baud rate 115200bps, this command enable broken line graph display speed up to maximum 5500 pixel per second

3.18.7 Scaling broken line graph in scale using data in temporary buffer（0xC105）

Tx: AA C1 05 <Address> <x> <y> <Line_Number> <D_x> <Dis_x> <M_y> <D_y> <Color> CC 33 C3 3C

Rx: None

- <Address>: the starting address, word address, in temporary buffer(RAM); totally 40k word, ranging from 0x0000 to 0x9FFF.
- <x>: the starting coordinate in X-axis.
- <y>: the zero position(the lowest point) in Y-axis; actual pixel position equals to y-Ly.
- <Line_Number>: number of lines with maximum 40960, ranging from 0x0000 to 0x9FFF. Every line takes up 1 word unit.
- <D_x>: pixels' distance in read buffer, ranging from 0x01 to 0xFF;i.e.Address=Address+D_x after connecting.
- <Dis_x>: the increment in X-axis, ranging from 0x01 to 0x0F;i.e.x=x+Dis_x after connecting.
- <M_y> <D_y>: amplification factor of Y-axis, ranging from 0x00 to 0xFF; and height equals to Y×M_y/D_y,e.g. M_y=4, D_y=2 means amplify Y-axis twice.
- <Color>: color of lines without any variation in system palette.

The line data format in temporary buffer is defined as Ly, 2 bytes, which is the height of pixel.

3.18.8 Display window limit two-way broken line graph in scale using data in temporary buffer (0xc106)

Tx: AA C1 06 <Address> <x> <y> <Line _Number> <D _x> <Dis _x> <M _y> <D _y>

<Color> <Ymin> <Ymax> CC 33 C3 3C

Rx: None

<Address> the starting address, word address, in temporary buffer (RAM); totally 40k word, ranging from 0x0000 to 0x9FFF.
<x> x coordinates of the starting position
<y> y coordinates of 0 point position, actual connection point=y+Ly or y-Ly;
<Line _Number> number of connection line ,0x0000-0x9FFF,each line is one word ,maximum 40960 line;

<D _x> read the space between the point in the temporary buffer,0x01—xFF, i.e connect one line ,Address=Address + D _x;
<Dis _x> increase of X coordinate, 0x01-0x0F,i.e connect one line ,x=x+Dis_x;
<M _y> <D _y> y coordinate amplification factor, 0x00-0xFF,dislay height=y*M_y/D_y, e.g. M_y=4,D_y=2, the y coordinate amplification factor is 2;
<Color> the color of the line display, do not alter the palette's settings.
<Ymin> minimum of Y coordinate (window's minimum)
<Ymax> maximum of Y coordinate(window's maximum)
The line data format in temporary buffer is defined as Ly, 2 bytes .Instruction as follow:
Ly.15 i.e direction of connection, 0=positive ,1=negative; e.g. Ly=0x8010 i.e negative height 16;
Ly.14-Ly.0 i.e height of connection.

3.18.9 Buffer command to realize display synchronously using temporary buffer(0xC110)

Tx: AA C1 10 <Address> <Frame _Number> CC 33 C3 3C
Rx:None
<Address> the starting address, word address, in temporary buffer(RAM); totally 40k word, ranging from 0x0000 to 0x9FFF.
<Frame _Number> number of command frame ,0x01-0xFF;
Definition of command frame in the temporary buffer as follow:

Starting Address	Definition	Frame Structure
<Address>	The first command frame	frame length +command +data e.g. display the image 0,definition is 02 70 00
<Address>+0x80	The second command frame	
……		
<Address>+0x80×(K-1)	K command frame	

Instruction:
Because data is saved by word in the temporary buffer, the address which one command frame occupy is 0x80,and actual memory space is 256 byte.

3.19 Keyboard and I/O Operation(0x70,0x71,0x34,0x35,0x36)

3.19.1 Key code upload（0x70）

Tx: None
Rx: AA 70 <K_code> CC 33 C3 3C

- <K_code>: user pre-defined 2 byte key codes, uploaded key code automatically to MCU when HMI's keyboard interface detects the release action

3.19.2 Key code upload（0x71）

Tx: None
Rx: AA 71 <K_code> CC 33 C3 3C

- <K_code>: user pre-defined 2 byte key codes, uploaded key code automatically to MCU when HMI's keyboard interface detects the pressing action

3.19.3 I/O port setting(0x34)

Tx: AA 34 <DIR[15:8] , DIR[7:0]> CC 33 C3 3C
Rx: None

- < DIR[15:8] , DIR[7:0]>: 2 bytes parameter, 16 significant bits, correspond to setting of 16 I/O port direction respectively; "0" mean input, "1" mean output; The setting of port direction will be of no effect if the pins is occupied by keyboard .

3.19.4 Write I/O port command (0x35)

Tx: AA 35 <DIR[15:8] , DIR[7:0]> CC 33 C3 3C
Rx: None
< DIR[15:8] , DIR[7:0]>: 2 bytes parameter, 16 significant bits, correspond to 16 I/O port respectively; The writing value will be of no effect if the pins is occupied by keyboard or defined by input pin.

3.19.5 Read I/O port command (0x36)

Tx: AA 36 CC 33 C3 3C
Rx: AA 36 <DIR[15:8] , DIR[7:0]> CC 33 C3 3C
< DIR[15:8] , DIR[7:0]>: 2 bytes parameter, 16 significant bits, correspond to 16 I/O port respectively; The reading value will be of no effect if the pins is occupied by keyboard or defined by output pin.

Keyboard interface and GPIO port:

A. Introduction

The Keyboard interface and GPIO port share a common socket of 18 pins, include 16 I/O port inside; And the packaging of the socket is 2.54mm equal distance.

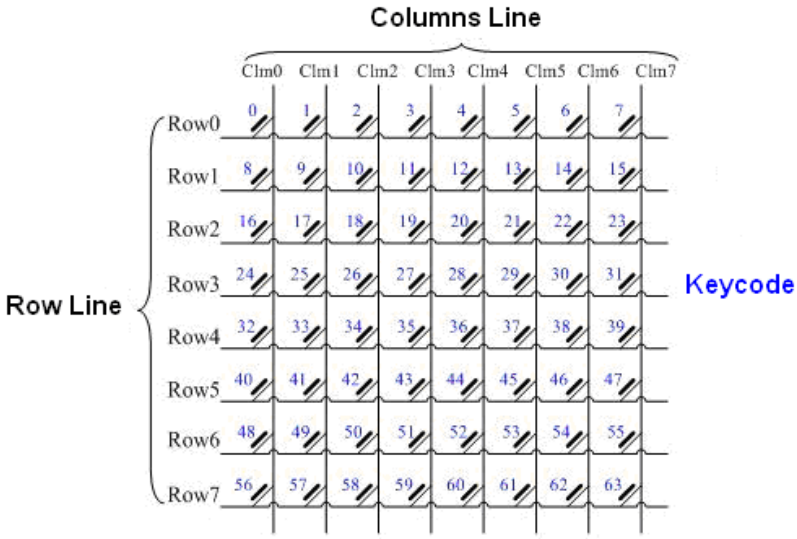
There are 3 application mode by the 18 pins. (Define by “0xE0” command)

- 8x8 matrix keyboard: the Maximum of key is 64. See Table-01 below.
- 4x4 matrix keyboard & 8 I/O ports: the Maximum of key is 16. See Table-02 below.
- 16 I/O ports. See Table-03 below.

Table-01

Pin code	Name	Function		Pin code	Name	Function
1	Row0	Keyboard Row Line 0		10	Clm1	Keyboard columns Line 1
2	Row1	Keyboard Row Line 1		11	Clm2	Keyboard columns Line 2
3	Row2	Keyboard Row Line 2		12	Clm3	Keyboard columns Line 3
4	Row3	Keyboard Row Line 3		13	Clm4	Keyboard columns Line 4
5	Row4	Keyboard Row Line 4		14	Clm5	Keyboard columns Line 5
6	Row5	Keyboard Row Line 5		15	Clm6	Keyboard columns Line 6
7	Row6	Keyboard Row Line 6		16	Clm7	Keyboard columns Line 7
8	Row7	Keyboard Row Line 7		17	VCC	Power

Picture-01



Picture-02

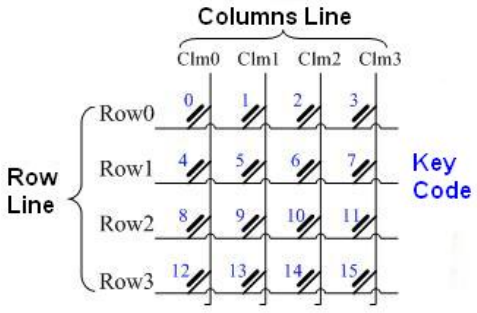


Table-02

Pin code	Name	Function		Pin code	Name	Function
1	Row0	Keyboard Row Line 0		10	GPIO9	I/O port 9
2	Row1	Keyboard Row Line 1		11	GPIO10	I/O port 10
3	Row2	Keyboard Row Line 2		12	GPIO11	I/O port 11
4	Row3	Keyboard Row Line 3		13	GPIO12	I/O port 12
5	Clm0	Keyboard columns Line 0		14	GPIO13	I/O port 13
6	Clm1	Keyboard columns Line 1		15	GPIO14	I/O port 14
7	Clm2	Keyboard columns Line 2		16	GPIO15	I/O port15
8	Clm3	Keyboard columns Line 3		17	VCC	Power
9	GPIO8	I/O port 8		18	GND	Ground

Table-03

Pin code	Name	Function		Pin code	Name	Function

1	GPIO0	I/O port 0		10	GPIO9	I/O port 9
2	GPIO1	I/O port 1		11	GPIO10	I/O port 10
3	GPIO2	I/O port 2		12	GPIO11	I/O port 11
4	GPIO3	I/O port 3		13	GPIO12	I/O port 12
5	GPIO4	I/O port 4		14	GPIO13	I/O port 13
6	GPIO5	I/O port 5		15	GPIO14	I/O port 14
7	GPIO6	I/O port 6		16	GPIO15	I/O port15
8	GPIO7	I/O port 7		17	VCC	Power
9	GPIO8	I/O port 8		18	GND	Ground

B. Key Code Table.
For 8x8 matrix keyboard, there are 64 cross points, each cross point has one key code, from 0 to 63.
Default key code for 8x8 matrix keyboard:

0,1,2,3,4,5,6,7,
8,9,10,11,12,13,14,15,
16,17,18,19,20,21,22,23,
24,25,26,27,28,29,30,31,
32,33,34,35,36,37,38,39,
40,41,42,43,44,45,46,47,
48,49,50,51,52,53,54,55,
56,57,58,59,60,61,62,63;

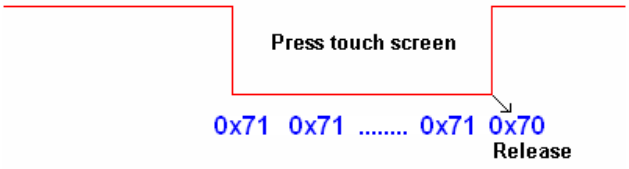
For 4x4 matrix keyboard, there are 16 cross points, each cross point has one key code, from 0 to 15.
Default key code for 4x4 matrix keyboard:

0,1,2,3,
4,5,6,7,
8,9,10,11,
12,13,14,15;

User can define any key code by ".txt" file, then download this application file into terminal by Assistant software. (Download Key Config File option)
Please note that between of numbers should be divided with "," and it should add ";" behind the last number.

C. Key Board Command.

Command	Parameter	Instruction
0x71	Key_Code	uploaded key code as press action
0x70	Key_Code	uploaded key code as release action
0xE0	Baud_Rate+Para1+Para2	Setting Working Mode Para1+Para2 Same with touch mode.



3.20 Read and Write in User's Memory（0x90，0x91）

the physical media of database is NAND Flash with 100 thousand erasable times and 10 years life circle.

3.20.1 write in random data memory（0x90 64KB）

Tx: AA 90 55 AA 5A A5 01 DE <Address> <Data0……Datai> CC 33 C3 3C
Rx: AA 90 4F 4B CC 33 C3 3C

- <Address>: the starting address to write in data memory, 2 bytes, ranging from 0x0000 to 0xFFFF.

- <Data0……Datai>: 1 data series to be written.

3.20.2 write in sequential data memory（0x90 30MB）

Tx: AA 90 55 AA 5A A5 <Address> <Data0……Datai> CC 33 C3 3C
Rx: AA 90 4F 4B CC 33 C3 3C

- <Address>: the starting address to write in data memory, 4 bytes, ranging from 0x00000000 to 0x01DDFFFF.
- <Data0……Datai>: 1 data series to be written.

Different from random data memory, sequential data memory can only write sequentially, i.e. It cannot write randomly.

*For display terminal with resolution of 640*480 and above, the whole sequential memory is divided into 120 256KB data page. When meet with the head of page (address=* *******0 00 00),it will erase current page with no data backup before erasing and no effect on other pages. It is recommended to do some continuous data of large amount storage such as paperless recording and audio recording.*

*For display terminal with resolution of 480*272 and below, the whole sequential memory is divided into 240 128KB data page. When meet with the head of page (address=* *******0 00 00),it will erase current page with no data backup before erasing and no effect on other pages. It is recommended to do some continuous data of large amount storage such as paperless recording and audio recording.*

3.20.3 read data memory（0x91）

Tx: AA 91 <Address> <Length> CC 33 C3 3C
Rx: AA 91 <Address> <Length> <Data0……Datai> CC 33 C3 3C

- <Address>: the starting address to read in data memory, 4 bytes, ranging from 0x00000000 to 0x01DDFFFF;
  RAM address range :0x01DE0000-0xDEFFFF
- <Length>: the length of data memory to read, 2 bytes. Most read 64KB once.
- <Data0……Data I>: 1 data series to read out.

3.20.4 Copy the 64K random data to the sequential storage area (Note: Only the V6.0 and its later versions support this command)

The storage medium is NAND Flash, erasable for 100,000 times with 10 years life span.

Tx: AA 92 55 AA 5A A5 <Address> CC 33 C3 3C
Rx: AA 92 4F 4B CC 33 C3 3C

- The <Address> indicates the first address of sequence data memory with 4 bytes, 0x00000000-0x01DE0000;
  Address must be multiples of 64K, otherwise the command will not execute.

3.20. 5 Write backup memory (0x93)

The storage medium is SRAM, the built-in backup battery will save data when power off, without limit of erasing times and life span, quite suitable for occasions that need frequent modification. The total is 9 words, 18 bytes, the reading and writing length are both double bytes as a unit.

Tx: AA 93 <Address> <Data0……Datai> CC 33 C3 3C
Rx: AA 93 4F 4B CC 33 C3 3C

- The <Address> indicates the first address of writing backup memory, 1 bytes, 0x00-0x08;
- <Data0……Datai> indicates the data string, the length must be even number bytes and shall not exceed the capacity of backup volume.

3.20.6 Read backup register (0x94)

Tx: AA 94 <Address> <Length> CC 33 C3 3C
Rx: AA 94 <Address> <Length> <Data0……Datai> CC 33 C3 3C

- The <Address> indicates the first address of reading the backup memory, 1 bytes, 0x00-0x08;
- The <Length> length read; 1 byte; double bytes as a unit, shall not exceed the capacity of the backup volume.
- The <Data0 / Datai> data strings read, the form of the word.

Note: the capacity of the backup register is 9 words (18 bytes), and its reading and writing are all in the form of word (2 bytes).

3.21 Virtual Keyboard (0x7C)

Virtual keyboard-touch screen (number, Chinese and English input)

Virtual keyboard can help facilitate the Chinese, English and number input for the user.

After activating the virtual keyboard, the display terminal does not respond to any of the user's command. When the user touches the 'Back' button on the virtual keyboard, the display terminal would quit the virtual keyboard automatically and return the ASCII code or Chinese GBK code.

3.21.1 variables or password input

1. without echo function

This command sends the data string input directly to the user's MCU.

Tx: AA 7C 01 <X> <Y> <Str_MaxNum ><Str_Scal > CC 33 C3 3C

Rx: AA 7C 01 <Str_Number ><String > CC 33 C3 3C

- < X >< Y >: 2 byte parameter, the top left coordinate of Virtual Keyboard;
- < Str_MaxNum >: 1 byte parameter, Define the max length of String, the max length is 15 numbers;
- < Str_Scal >: 1 byte parameter.

Str_Scal [7]	0: Display the number normally 1: Display "*" to replace the input of number (Password Input)
Str_Scal [6]	0: Don't clear the screen when exit 1: Clear the screen
Str_Scal [5:0]	0

- <Str_Number >: 1 byte parameter, Return the length of String that user practically input, 0x00 mean the void input
- <String >: its length is decided by the Str_Number, Return the ASCII code of the numbers that user practically input.

eg1: input variables via virtual keyboard

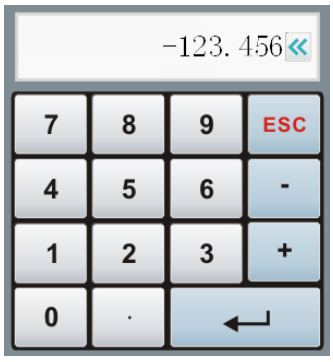
Send: AA 7C 01 00 0a 00 0a 0C 03 CC 33 C3 3C

The display terminal switches to variable input mode, see pic 9-1-1.

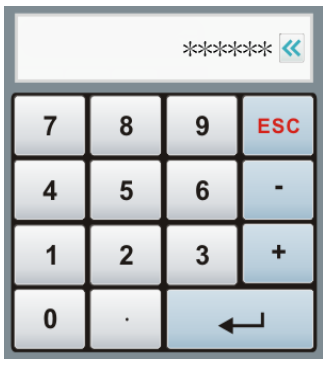
eg 2: input password via virtual keyboard

Send: AA 7C 01 00 0a 00 0a 0C 83 CC 33 C3 3C

The display terminal switches to password input mode, see pic 9-1-2.



9-1-1 variable input



9-1-2 password input

2. with echo function

This command sends the data string input to the user's MCU and shows them at the designated position of the screen.

Tx: AA 7C 04 <X> <Y> <Str_MaxNum> <Str_Scal> <T_X> <T_Y> <Lib_ID> <C_Mode> <C_dots> <Fcolor> <Bcolor> CC 33 C3 3C

Rx: AA 7C 04 <Str_Number> <String> CC 33 C3 3C

□ □ <X> <Y>: 2 byte parameter, the top left coordinate of Virtual Keyboard;

<Str_MaxNum>: 1 byte parameter, Define the max length of String, the max length is 16 numbers;

□ □ <Str_Scal> 1 byte parameter;

Str_Scal [7]	0: Display the number normally 1: Display "*" to replace the input of number (Password Input)
Str_Scal [6]	0: Don't clear the screen when exit 1: Clear the screen
Str_Scal [5:0]	0

<T_X> <T_Y> coordinates of the top left corner of the echo data string, 2 bytes parameter.
<Lib_ID> <C_Mode> <C_dots> <Fcolor> <Bcolor> the same as 0x98 command
<Str_Number >: 1 byte parameter, Return the length of String that user practically input, 0x00 mean the void input
<String >: its length is decided by the Str_Number, Return the ASCII code of the numbers that user practically input.

3.21.2 Chinese, English, Number Input

1. Without echo function
This command sends the data to MCU directly.
Tx: AA 7C 02<X> <Y> <Str_MaxNum ><Str_Scal > CC 33 C3 3C
Rx: AA 7C 02 <Str_Number ><String > CC 33 C3 3C
< X >< Y >: 2 byte parameter, the top left coordinate of Virtual Keyboard;
< Str_MaxNum >: 1 byte parameter, Define the max length of String, the max length is no more than 32 Characters;

Str_Scal [7]	0: Display the number normally 1: Display "*" to replace the input of number (Password Input)
Str_Scal [6]	0: Don't clear the screen when exit 1: Clear the screen
Str_Scal [5:0]	0

<Str_Number >: 1 byte parameter, Return the total sum of ASCII code and GBK code (each number or character is equal to 1 ASCII code, each Chinese character is equal to 2 GBK codes) equivalent to the length of String that user practically input, 0x00 mean the void input
<String >: Its length is up to the Str_Number, return the ASCII code and GBK code of the characters or numbers that user practically input.
eg: realize input of Chinese, English and numbers via virtual keyboard
send AA 7C 02 00 0a 00 0a 0C 03 CC 33 C3 3C
The display terminal will switch to mix input mode of Chinese, English and numbers. See pic 9-1-3.



pic 9-1-3: mix input of Chinese, English and numbers
2. with echo function
This command sends the data string to MCU, and show the character on the specified area of the screen.

Tx: AA 7C 05 <X> <Y> <Str_MaxNum ><Str_Scal > <T_X> <T_Y> <Lib_ID> <C_mode> <C_dots> <F color> <B color> CC 33 C3 3C

Rx: AA 7C 05 <Str_Number ><String > CC 33 C3 3C

< X >< Y >: 2 byte parameter, the top left coordinate of Virtual Keyboard；

< Str_MaxNum >: 1 byte parameter, Define the max length of String, the max length is 16 numbers；

< Str_Scal >: 1 byte parameter.

Str_Scal [7]	0: Display the number normally 1: Display "*" to replace the input of number (Password Input)
Str_Scal [6]	0: Don't clear the screen when exit 1: Clear the screen
Str_Scal [5:0]	0

< T_X >< T_Y >: 2 byte parameter, the top left coordinate of the first character of the echoed string on the screen；

<Lib_ID> <C_mode> <C_dots> <F color> <B color> See 0x98 command.

<Str_Number >: 1 byte parameter, Return the length of String that user practically input, 0x00 mean the void input

<String >: Its length is up to the Str_Number, return the ASCII code and GBK code of the characters or numbers that user practically input.

3.21.3 Compulsively quit the virtual keyboard control

This command can quit the virtual keyboard compulsively.

Tx: AA 7C FF CC 33 C3 3C

Rx: None

3.21.4 Calculator

Tx: AA 7C 03 <X> <Y> CC 33 C3 3C
Click "Enter"
Rx: AA 7C 03 < Calculated_Value > CC 33 C3 3C
Click "ESC"
Rx: AA 7C 00 00 00 00 CC 33 C3 3C

- < X >< Y >: 2 byte parameter, the top left coordinate of Virtual Keyboard；
- < Calculated_Value >: Hex code is multiplied 100 times；

3.22 Menu & List Control (0x7D)

The menu/list control can help the user realize the function choosing interface of windows style.

The menu/list control requires the display terminal to be a touch screen.

After activating the menu/list control, the display terminal will not respond to any of the user's command. When the user selects a row of the control by touch screen, the display terminal will quit the virtual keyboard automatically and return the current serial number and the selected row number. If the user does not touch the valid area, the display terminal will also quit the virtual keyboard and return the fixed row value: 0XFF.

Tx：AA 7D <Lst_Num> <Para> CC 33 C3 3C

Rx：AA 7D <Lst_Num> <Index> CC 33 C3 3C

□ □ <Lst_Num> ： 2 bytes parameter, define the serial No of the menu/list control；

□ □ <Para>：1 byte parameter；

Para[7:2]	0

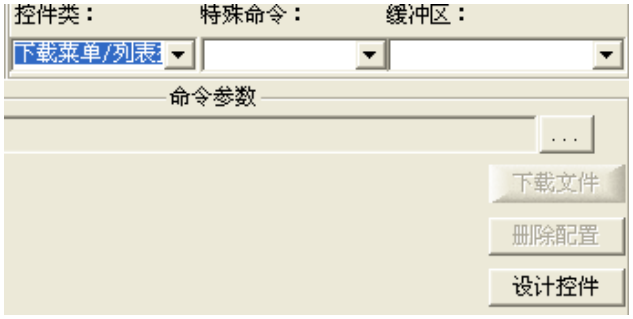
Para[1]	Define whether or not to add the divider when displaying the control 1：add divider；0：no divider
Para[0]	Define whether or not to recover the background when displaying the control 1：recover background；0：not recover the background.

□ <Index>：1 byte parameter, return the currently selected row number of the menu/list control. If the return value is 0xFF, it means the user touches the invalid area and quit the control.

Eg: make a pull-down menu/list control including ‘serial number’, ‘company name’, ’contact name’, ‘Bank of deposit’ and ‘City’.

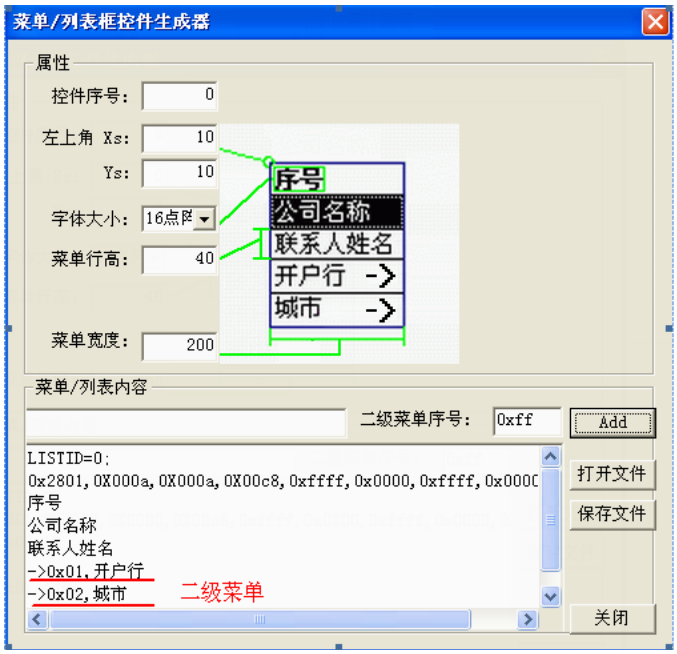
Step 1: generate the menu/list control document

Open ‘SDW intelligent display terminal demonstration developing assistant.exe’, select ‘controls’-‘download menu/list control’- ‘design control’ as shown in the picture 9-3-1.



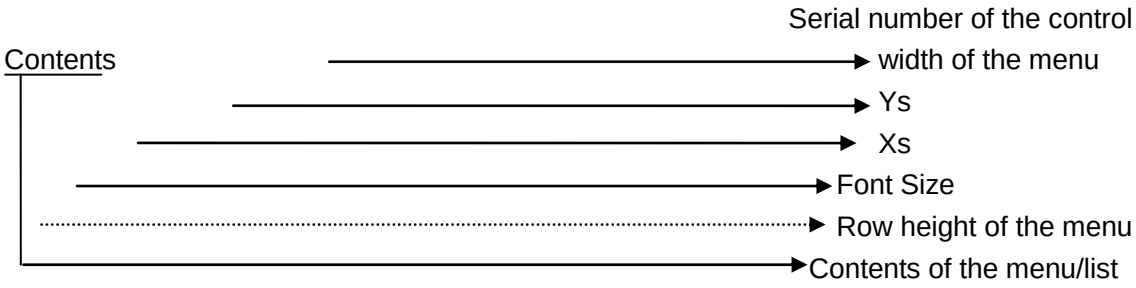
Picture 9-3-1

The dialogue box of ‘menu/list control generator’ will pop out. Fill in the relevant content and save the file as in the picture 9-3-2.



Picture 9-3-2

Eg: LISTID=0;
0x2801,0X0000,0X0000,0X00c8,0xffff,0x0000,0xffff,0x0000,0x0000;



Description of the menu/list content: if the ‘secondary menu control serial number’ is 0xff, it means there is no secondary menu. If the if the ‘secondary menu control serial number’ is not 0xff, then it means there is secondary menu and the ‘-> secondary menu control serial No.’ is added before the menu content as in the picture 9-3-2.

Note： The 16 dots matrix data string is recommended for the contents of the Menu/list control. The row height should be not less than 40 to facilitate touch operation. The contents of the menu cannot exceed 14 characters.

Step 2: download the menu/list control document

Open ‘SDW intelligent display terminal demonstration developing assistant-V3.13’, select ‘controls’-‘download menu/list control’ as shown in the following picture.



Picture 9-3-3

Step 3: call menu/list control

Eg: send AA 7D 00 00 03 CC 33 C3 3C

Check the menu result in the following picture:



Picture 9-3-4

Note: each menu cannot exceed 16 rows. The intelligent display terminal will not respond to any command until the user touches the screen to quit the control.

3.23 Buzzer Control（0x79）

Tx: AA 79 <On_Time>CC 33 C3 3C

Rx: None

➢ <On _Time> 0x01-0xFF, time to buzzer beepers, unit 10ms
The duration of buzzer's ringing.

3.24 Clock(RTC) Display and Read off（0x9B，0xE7）

3.24.1 Close clock display

Tx: AA 9B 00 CC 33 C3 3C
Rx: None

3.24.2 open clock display

Tx: AA 9B FF<RTC_Mode> <Text_Mode> <Color> <X> <Y> CC 33 C3 3C
Rx: None

➢ <RTC_Mode>: clock display mode setting
0x00: HH:MM:SS
0x01: 20YY-MM-DD HH:MM:SS
0x02: HH:MM:SS
0x03: HH:MM
0x04: YY-MM-DD HH:MM:SS
0x05: YY-MM-DD HH:MM
0x06: MM-DD HH:MM
0x07: Custom clock format

➢ The <Text_Mode> clock display font (no significant when RTC_Mode=0x07);
0x01:8*16 0x02:12*24 0x00:6*12
0x04:24*48 0x03:16*32
The <Color> clock display font color (no significant when RTC_Mode=0x07);
The <X> <Y> clock display position (no significant when RTC_Mode=0x07);
The custom clock format allows the independent display of year, month, day, hour, minute, second.

Before using the custom clock format, a custom clock file needs to be created and downloaded into the display terminal. The operating steps are as below:

1. The "development assistant - > special command - > Download custom clock file - > create a file", open the custom clock file, as following picture shown:



2. Choose the font size and color.
3. Set the date, time and week's display position and format in left upper corner coordinates
4. Click "create a file", and exit.
5. Though "development assistant - > special command - > Download custom clock file - > download files", the custom clock file will be downloaded to the display terminal.

3.24.3 clock setting

Tx: AA E7 55 AA 5A A5 <YY:MM:DD:HH:MM:SS> CC 33 C3 3C
Rx: None

➢ <YY:MM:DD:HH:MM:SS>: time format to be set as year : month :day : hour : minute : second, represented by BCD code.

e.g.
AA E7 55 AA 5A A5 08 0B 1C 0C 39 00 CC 33 C3 3C
to set current time to 12:57:00, Nov. 28th, 2008.

3.24.4 read off current time (the Gregorian calendar)

Tx: AA 9B 5A CC 33 C3 3C

Rx: AA 9B 5A <YY: MM: DD: WW: HH: MM: SS> CC 33 C3 3C

- <YY:MM:DD:WW:HH:MM:SS>: current clock data
  08 0C 19 04 09 3A 00, for example, represents 09:58:00, Thursday, Dec. 25th, 2008.
  If HMI do not support clock function, it will return unknown results.

3.24.5 Open Timer

Tx: AA 9B 5A <Para1> <Space> CC 33 C3 3C
Rx: AA 9B 5A <YY: MM: DD: WW: HH: MM: SS> CC 33 C3 3C

- Para1[0]: =1 immediately timed
  =0 start timing from zero second.

Para1[7:1]: undefined

- The <TimerH> <TimerL> time, unit: second.

Description:

When Para1[0]=1, the display terminal will send the current time immediately, and send the current time for once every Timer second;

When Para1[0]=0, the display terminal will send the current time when the clock's second is 0, and send the current time for once every Timer second.

3.24.6 Close Timer

Tx: AA 9B A0 CC 33 C3 3C
Rx: None

3.25 Configuration File - touchpad interface switch

DISPLAY MODULE can realize simplified Operating System function through following configuration files, which can reduce considerable codes.

DISPLAY MODULE, touchpad included, can download configuration file beforehand, which can reduce the code scale; and set touchpad interface to auto-switch mode as to realize user intervention-free function.

Developing process:

Step One: design a user interface of identical physical resolution of HMI and download it to HMI terminal.

Step Two: generate configuration file

Configuration file is a binary file with maximum 8192 touchpad commands. Every command takes up 16 bytes, defined as follows:

Starting address	Length （Byte）	Definition	Description
0x00	2	Pic_Now	serial number of the picture display on current screen if high byte of Pic_Now is 0xFF, it means touchpad command end.
0x02	4	x_s,y_s	left top coordinate of effective touchpad area
0x06	4	x_e,y_e	right bottom coordinate of effective touchpad area
0x0A	2	Pic_Next	switch to next picture serial number when press the effective area. if high byte of Pic_Next is 0xFF, it means no switch allowed.
0x0C	2	Pic_Cut	serial number of touchpad animate picture if high byte of Pic_Cut is 0xFF, it means no more touchpad animate pictures.
0x0E	2	Touch_Code	upload touchpad code when press the effective area(act as a message to trigger users' soft) if high byte of Touch_Code is 0xFF, it means no more uploading. if high byte of Touch_Code is 0xFE, it means the uploading data serial indexes to 0x1A configuration file where low byte of Touch_Code means the index ID. In the 0x1A configuration file, every index length is fixed to 256 bytes with the first one is the effective length.

Table 3-25-1 definitions of touchpad interface configuration file

The process of generating configuration files is the process in which users arrange the interface-switch procedures, which graphic designer usually do. Configuration files can be compiled by

UltraEdit or some kinds of compile system, such as C51 and ASM51.

The example showed in figure 3-25-1 represents how to compile a configuration file with the classic ASM51 compiler in DOS system, which is also the most familiar with engineers.





The above example above only compiles one touchpad key and one interface switch. More interface switches need users' more commands.

compile finished configuration file--*.ASM to HEX file, using compiler ASM51; then transform HEX file to BIN file which is the right configuration file.

Step 3: download the configuration file to HMI terminal.

Download the generated .txt format configuration file to library in HMI terminal.

Step 4: set HMI to touchpad interface auto-switch mode

set the 5th bit of Para1--Para1.5,0x20 to 1, using command 0xE0. When users press the screen, HMI terminal will automatically switch touchpad interface and upload user pre-defined key code instead of uploading coordinates.

Step 5: for testing whether interface switch works well, step 2 and 3 may need repeat times.

Definition of configuration file

configuration file consists of maximum 1024 commands with a maximum of 127 bytes per command. It occupies 128 bytes in memory. Commands definitions are represented as follows:

Starting address	Data length （Byte）	Definition	Description
0x00	1	Length_Command	The length of this command. If Length_Command=0x00, it represents that this command is a combination of commands--batch command.
0x01	indefinite	command	the command to be send first if Length_command is not 0x00 or 0x01.

		Command pointer （Cmd_EN,Cmd_ID,Tx_Delay）	if Length_Command＝0x00, what behind is the pointer of combination command. Each pointer takes up 4 bytes with a maximum 31 command-pointers, combination command. Pointer definitions, as follows: Cmd_EN: 0x00 represents the end of combination command sending; and others represent command sent. Cmd_ID：ID of commands to be sent, ranging from 0x00 to 0x3FF, correspond to the non-combination command numbers in 0x1A; and it do not support nested of combination command. Tx_Delay: time intervals of each sending commands; unit is 0.1 second. when sending combination commands, HMI do not respond to any users and peripherals.

Above the development process could be achieved by another .Users can generate touchpad files and do simulation test on the computer by the “touch control software” offered by STONE. Ask the STONE for specific use of the software.

Designing touchpad interface with configuration files can not only largely reduce code amount, it can reduce the difficulty of development;more important, we hope this can change our regular minds about products designing and market development, as follows:

- **Completely separate products' algorithm and interface designing**; algorithm designing is the core competitiveness of a company, thus the precious research resources should not waste on large amount copious interface designing.
- **Products research can process in parallel**; not only the interface designing and algorithm can process in parallel, interface designing itself can as well by several graphic designer; because touchpad key code has a function of "trigger message", engineers can design and debug algorithm in parallel.
- **Improve products' reliability**; all users' programmes lie in the same parallel level in principal, thus function modules are independent and easy for testing.
- **Easy upgrade**; when product is stable, product's upgrade is basically the upgrade of interface while algorithm needs little.
- Using the configuration file method, it is easy to realize **interface customization**--more skins, because users only need to insert different interface in configuration file and the key code uploaded is the same.
- Largely reduce the developing time of a new product, which enhances market competitiveness. Following is an example that STONE develops a product for customers:
  a.Using 1 to 3 working days to confirm basic demands with customers.
  b.Using 1 to 2 working days, our graphic designer can provide the product's interface based on standard HMI; meanwhile, our structure engineers are ready to provide product's three-dimensional rendering for customers to confirm.
  c. After confirmation, our hardware engineers start to design PCB, structure engineers hand in the design to factories in Shenzhen for fast molding and graphic designers begin to modify interface as to submit software engineer the process documents.
  d.One week later, we can provide customers the full-fledged prototype for inspection.

With reference to the way of generating configuration file.

To match up configuration file, using command 0x99 can easily solve these problems:

- For analog dial, we can make different scales into icons and **directly** call it according to the variables instead of looking-up table again.
- For **special characters**, even those UNICODE does not contain, we can call it **in the way of icon**, which avoids the complex character library generating.
- It is convenient to use **Windows elements** such as **cursor** and **mouse pointer**.

3.26 Variable Display

3.26.1 Display integer Variable (0x1A 01)

Tx: AA 1A 01 <X> <Y> <Mode> <Fcolour> <Bcolour> <Int_Value> CC 33 C3 3C
Rx: None

- <X> <Y>: Coordinates of the left top corner of the first character, 2 byte parameter.
- <Mode>: Variables display mode, 4 byte parameter.

Mode	Mode[15:12] the most integer bit displayed 1 to 12 Mode[11:8] save Mode[7] 0: Don't show "0" in invalid integer bit 1: Display "0" in invalid integer For example: Mode[15:12]=3, Mode[7]=1, so it will show "3" in the form of "003". Mode[6] when transparently displaying, whether refresh the background image =1 refresh the background; =0 don't refresh the background Mode[5] 0: Foreground color not display 1: Foreground color display Mode[4] 0: Background color not display 1: Background color display Mode[3:0] Font size 0=6*12 1=8*16 2=12*24 3=16*32 4=24*48 5=32*64 6=48*96 7=64*128

- <Fcolour> <Bcolour>: Foreground colour & Background colour, 2 byte parameter.
- <Int_Value> 4 byte parameter, Display the Integer Variables value, range from -2147483648 to +2147483648.

e.g.
AA 1A 01 00 64 00 64 30 A2 FF FF 00 00 00 26 CC 33 C3 3C

Note: Split the Integer Variables value

```
int val=123456789
Unsigned long *p;
p = ( Unsigned long *)&val;     // Send High bit data firstly, then Send low bit data.
SBUF = (*p) >>24;               // Extract the First byte and send.
/*….*/                          // Other program.
SBUF = (*p) >>16;               // Extract the Second byte and send.
/*….*/                          // Other program.
SBUF = (*p) >>8;                // Extract the Third byte and send.
/*….*/                          // Other program.
```

```
SBUF = *p;                    // Extract the Fourth byte and send.
/*….*/                        // Other program.
```

Function:

// Entrance Condition: integer variable

// Exit Condition: 4 bit referenced variable bt, bt[3] is high-order word, bt[0] is low-order word

```
Void int2byte(int val, unsighned char *bt)
    { unsigned long *p;
      p=(unsigned long *)&val;
      *bt++=(*p)&0xff;
      *bt++=((*p)>>8)&0xff;
      *bt++=((*p)>>16)&0xff;
      *bt++=((*p)>>24)&0xff;
    }
```

3.26.2 Display Floats Variables (0x1A 02)

Tx: AA 1A 02 <X> <Y> <Mode> <Fcolour> <Bcolour> <Float_Value> CC 33 C3 3C

Rx: None

- <X> <Y>: Coordinates of the left top corner of the first character, 2 byte parameter.
- <Mode>: Variables display mode, 4 byte parameter.

	Bit	Definition	Description
Mode	Mode[15:12]	integer number bit	1 to 9
	Mode[11:8]	Decimal number bit	0 mean it has no Decimal number 1 to 6
	Mode[7]	0: Don't show "0" in invalid integer bit	For example: Mode[15:12]=3, Mode[7]=1, so it will show "0.3" in the form of "000.3".
		1: Display "0" in invalid integer	
	Mode[6]	None	
	Mode[5]	0: Fcolor not display	Foreground color
		1: Fcolor display	
	Mode[4]	0: Bcolor not display	Background color
		1: Bcolor display	
	Mode[3]	0: decimal point display width without compression 1: decimal point display width and automatically	V6.2 supports
	Mode[2:0]	Font size	0=6*12 1=8*16 2=12*24 3=16*32

- <Fcolor> <Bcolor>: Foreground color & Background color, 2 byte parameter.
- <Float_Value> 4 byte parameter, Display the Integer Variables value, range from -2147483648 to +2147483648.

e.g.

AA 1A 01 00 64 00 64 32 A2 FF FF 00 00 00 26 CC 33 C3 3C

Note: Split the Integer Variables value

```
Float val=123.456
Unsigned long *p;
p = ( Unsigned long *)&val;     // Send High bit data firstly, then Send low bit data.
SBUF = (*p) >>24;               // Extract the First byte and send.
```

```
/*….*/                    // Other program.
SBUF = (*p) >>16;         // Extract the Second byte and send.
/*….*/                    // Other program.
SBUF = (*p) >>8;          // Extract the Third byte and send.
/*….*/                    // Other program.
SBUF = *p;                // Extract the Fourth byte and send.
/*….*/                    // Other program.
```

Function:

```
// Entrance Condition: Float variable
// Exit Condition: 4 bit referenced variable bt, bt[3] is high-order word, bt[0] is low-order word
 Void int2byte(float val, unsighned char *bt)
     { unsigned long *p;
       p=(unsigned long *)&val;
       *bt++=(*p)&0xff;
       *bt++=((*p)>>8)&0xff;
       *bt++=((*p)>>16)&0xff;
       *bt++=((*p)>>24)&0xff;
     }
```

3.26.3 Variable Control File display (0x1A 03) --- Show huge mass of Variable

Tx: AA 1A 03 <Con_Name> <Val0 ….Vali > CC 33 C3 3C

Rx: None

- < Con_Name >: Sequence number of the Variables control file, 1 byte parameter.
- < Val0 ….Vali >: Display the Variable values, all of them are 4 byte parameter.

Note: It need to define and download the Variables control file by Assistant software firstly, otherwise "0x1A 03"command can't be used.

Variables Control file include:

Code	Name	Define	Description
1	Val_type	Variable type	01: Integer Variable 02: Float Variable
2	X	X axis coordinate	2 byte parameter
3	Y	Y axis coordinate	2 byte parameter
4	Mode	Variable display mode	See the Table below
5	Fcolour	Foreground colour	2 byte parameter
6	Bcolour	Background colour	2 byte parameter

Variable Display Mode define:

	Bit	Definition	Description
Mode	Mode[15:12]	integer number bit	1 to 9
	Mode[11:8]	Decimal number bit	0 mean it has no Decimal number 1 to 6
	Mode[7]	0: Don't show "0" in invalid integer bit	For example: Mode[15:12]=3, Mode[7]=1, so it will show "0.4" in the form of "000.4".
		1: Display "0" in invalid integer	

	Mode[6]	None	
	Mode[5]	0: Fcolor not display 1: Fcolor display	Foreground colour
	Mode[4]	0: Bcolor not display 1: Bcolor display	Background colour
	Mode[3:0]	Font size	0=6*12 1=8*16 2=12*24 3=16*32 4=24*48 5=32*64 6=48*96 7=64*128

For example:

VALID=0;

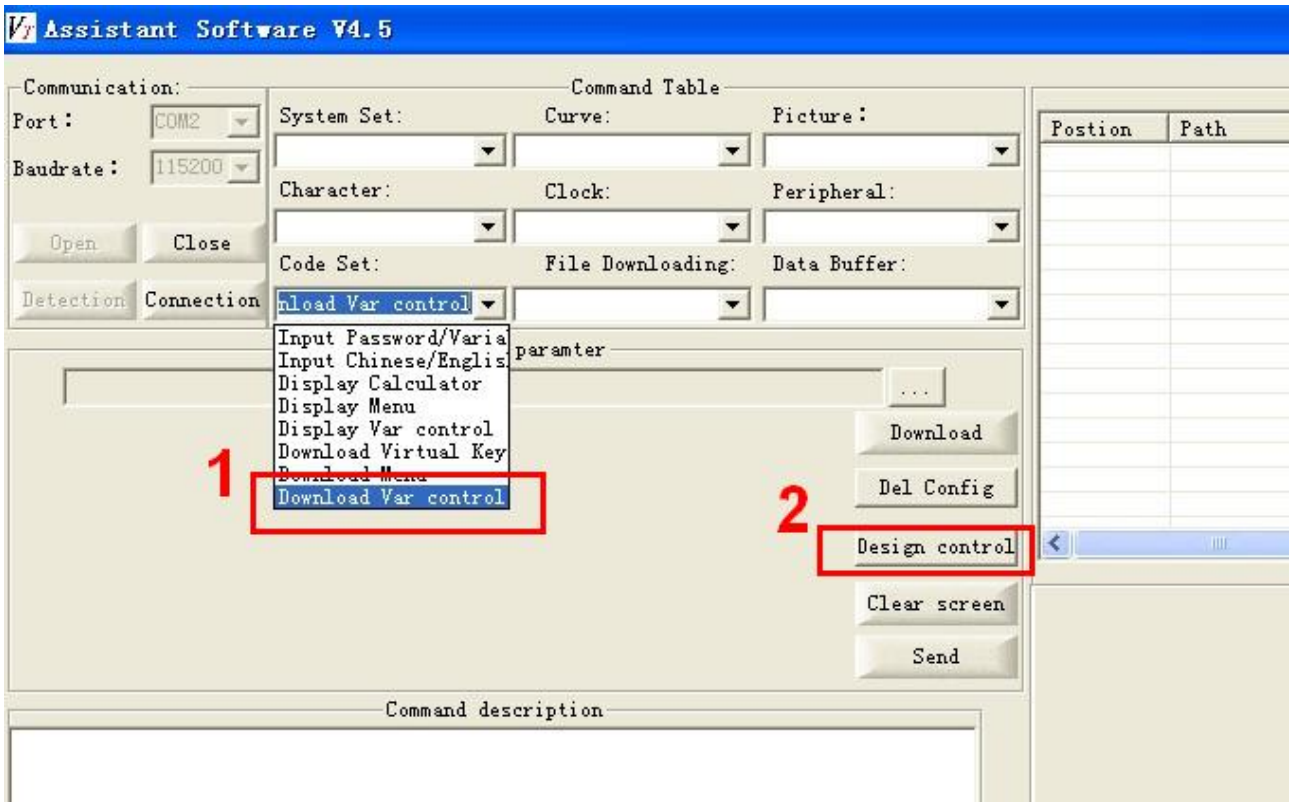
0x0001,0x0000,0x0000,0x6333,0xFFFF,0x0000. (0th Variables Control file)

VALID =1;

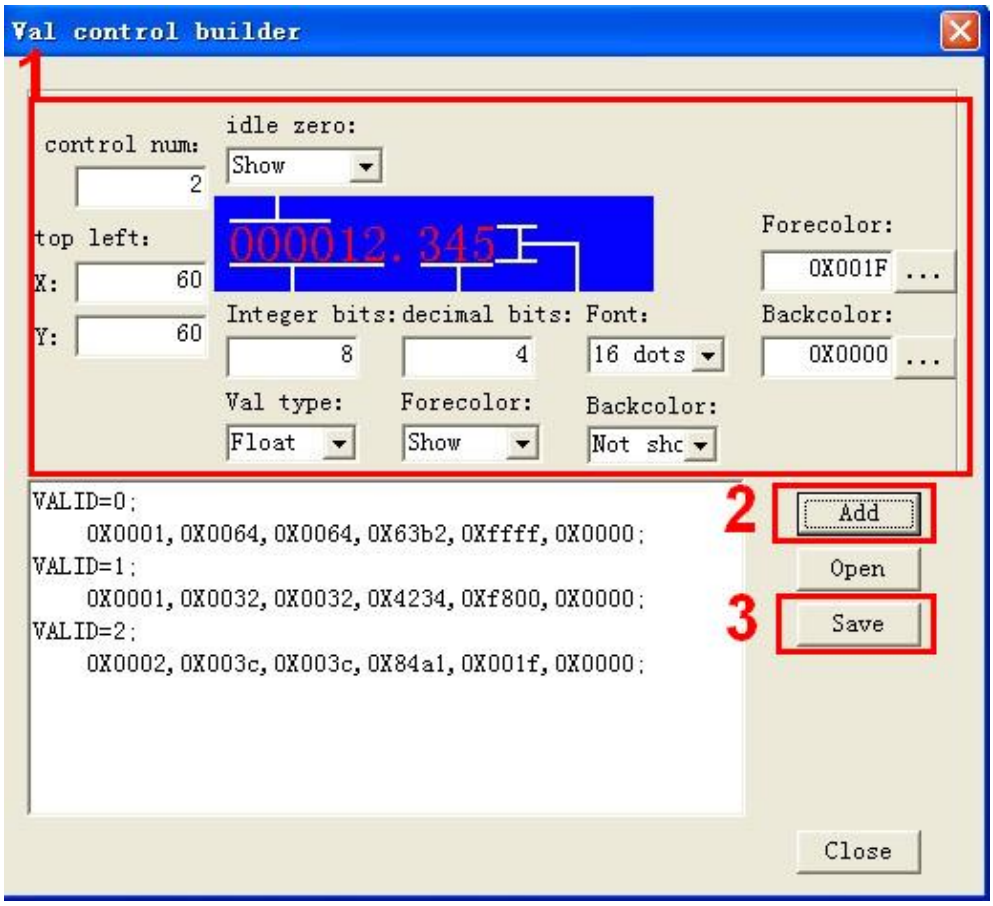
0x0002,0x0000,0x00C8,0x6333,0xFFFF,0x0000. (1th Variables Control file)

Build Steps:

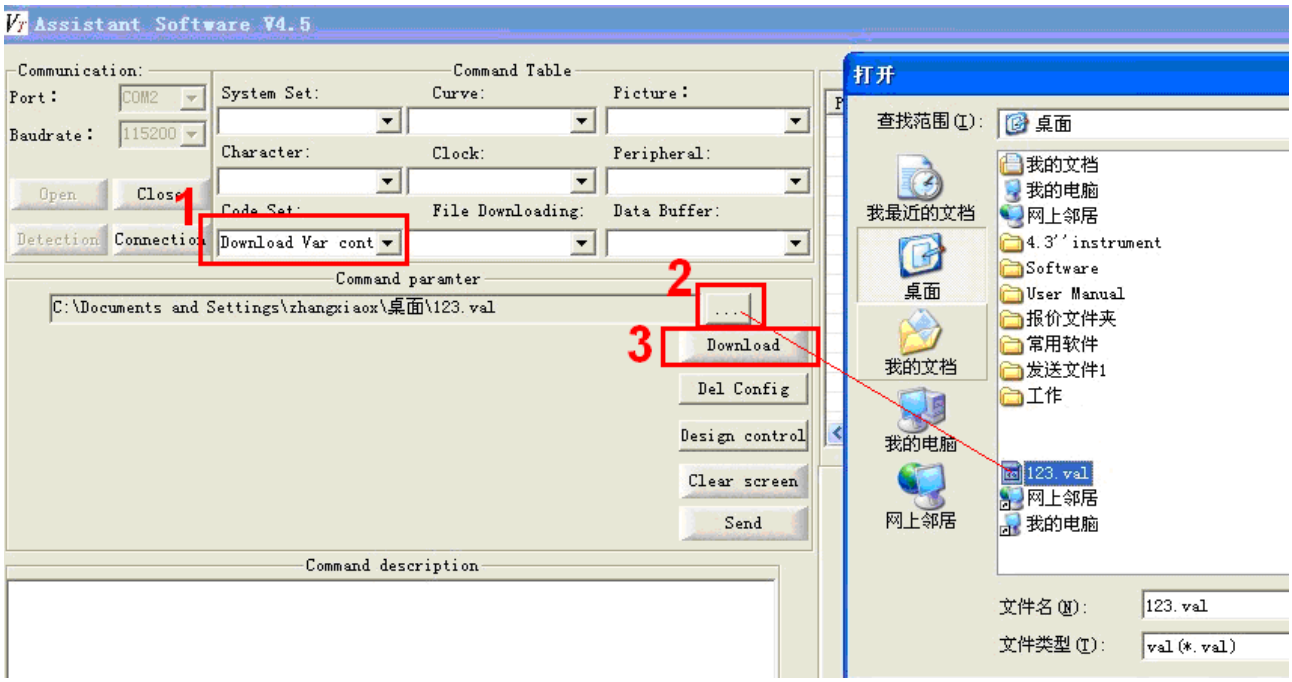
1. Open “Design control file” option



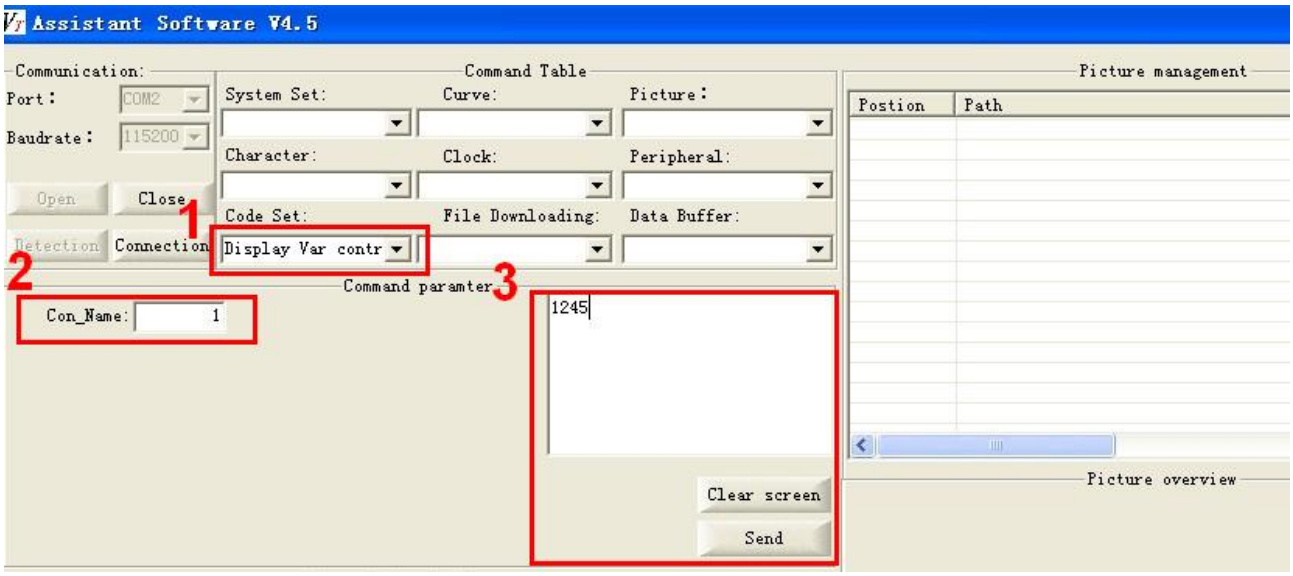
2. Define and Build the Variables Control file (.val format file)



3. Download the Variables Control file



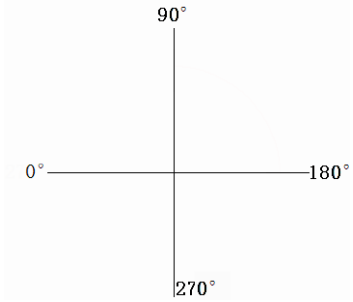
4. Test the Variables Control file



3.27 Dial Control

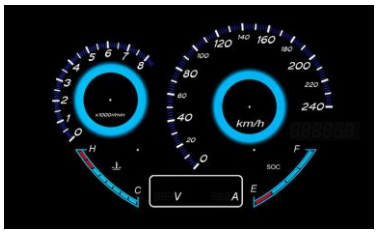
3.27.1 Dial Control Design

1.Set up the coordinate system



2. Picture Design

Step 1. Design the background picture for the dial



Step 2. Design the 0° pointer picture with pure background



Step 3. Generate files

1. Open the ‘SDW dial control design software’, assign the serial number and accuracy.
2. Click the ‘add picture’ button on the top left corner to add background picture; define the coordinates of the dial background, rotating center and the pointer’s swing range.



The coordinates of the dial background are decided by the red background frame on it. Position of the background frame can be moved via mouse and the arrow keys of the keyboard. The size of the background frame can be changed by dragging of the mouse. Also, the position and size of the background frame can be changed by the value of the coordinates.

The coordinates of the rotating center is defined by the cross lines on the background frame. The position of the cross lines can be moved via mouse or arrow keys of the keyboard. It can also be changed through changing the value of the input coordinates directly.

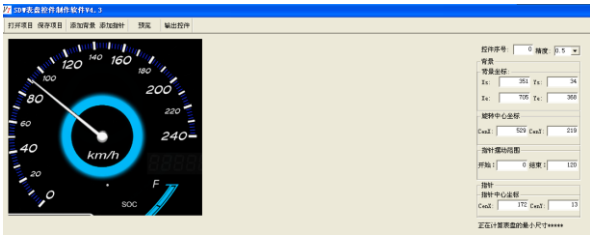
The dial pointer's swing range is decided by the starting and ending angle when it moves clockwise.

3. Click the 'add pointer' button on the top left corner to add the pointer picture and define the coordinates of the center of the pointer.



Like the rotating center of the dial background, the coordinates of the rotating center of the dial pointer is defined by the cross lines on the dial pointer picture. The position of these cross lines can be moved via mouse or arrow keys of the keyboard. It can also be changed through changing the value of the input coordinates directly.

4. Click the 'preview' button on the top left corner to have a preview of the dial board.

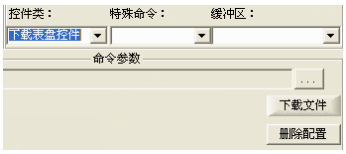


The actual rotating speed is much faster than the preview rotating speed on the display terminal.

5.click the 'control output' button on the top left corner to create a control file for the dial.

3.27.2 Download the dial control files

Open the 'SDW intelligent display terminal demonstration assistant', select 'control type'-'download the dial control files'.



3.27.3 Dial control commands(0x9E)

1 Close the dial control

Tx: AA 9E 00 <Con_Name> CC 33 C3 3C

Rx：None

□ □<Con_Name> control serial number，1 byte parameter.

Eg:AA 9E 00 00 CC 33 C3 3C；close dial control 0.

2 activate the dial control

Tx: AA 9E 01 <Con_Name> <StartAngl> CC 33 C3 3C

Rx：None

□ □ <Con_Name> Control serial number

□ □<StartAngl> starting angle of the control, unit is‘ 0.5°’

Eg: AA 9E 01 00 00 00 CC 33 C3 3C；activate dial control 0, starting angle is 0°.

3 display the specified angle of the dial control

Tx: AA 9E 02 <Con_Name> <EndAngl> CC 33 C3 3C

Rx：None

□ □ <Con_Name> dial control serial number, 1 byte parameter

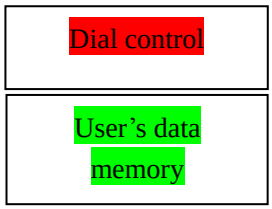
□ □ <EndAngl> 1 byte parameter

EndAngl [15]	For partial circle dial (fan-shaped), there is no definition to this, just set 0. For circular dial (similar with compass), this command specifies its rotating direction: 1. Counter-clockwise 2. Clock wise
EndAngl [14:10]	No definition to this, just set 0
EndAngl [9:0]	The ending angle（0-719），the unit is “0.5°”。

Eg：AA 9E 02 00 01 68 CC 33 C3 3C。 control 0 rotates to 180°position。

3.27.4 Storage position of dial control

The storage position of the dial control and the user’s sequential data memory overlaps, i.e. if the user uses the dial control and the sequential data memory at the same time, the size of the dial control needs to be checked and its sequential memory address needs to be calculated so as to avoid the dial control. Please check the following picture.



0x00000000

User Address

0x01DDFFFF

Eg: download a 4.25MB dial control with an intelligent display terminal of definition 640X480 or above, the starting and ending addresses of the sequential data memory available for the user are:

User_Address=(int(4.48*1024/256)+1)*256*1024=(17+1)*256*1024=0x480000

Thus, starting and ending addresses of the sequential data memory are 0x480000.

If download a 4.25MB dial control with an intelligent display terminal of definition 480X272 or below, the starting and ending addresses of the sequential data memory available for the user are:

User_Address=(int(4.48*1024/128)+1)*128*1024=(35+1)*128*1024=0x480000

Thus, starting and ending addresses of the sequential data memory are 0x480000.

3.28 Download Font file and Configuration Files by MCU（0xF2）

Tx: AA F2 F2 F2 5A A5 <Lib_ID> CC 33 C3 3C

Rx: 50 6C 65 61 73 65 20 54 78 20 54 65 78 74 5F 4C 69 62 20 21 (Please Tx Text_Lib !)

Then users can download the data of file into the specified storage position.

When there is no data for 100ms after finishing file saving,

Terminal will response: 2A 2A 2A 2A 2A 2A 4F 6E 65 20 54 65 78 74 5F 4C 69 62 20 53 61 76 65 64

20 4F 4B 20 21 2A 2A 2A 2A 2A 2A (******One Text_Lib Saved OK !******)

- <Lib_ID>: The storage position of Font file is from 0x00-0x0F, in which 0x00-0x07 is for small font libraries (less than 256KB font file) and 0x08-0x0F is for big font libraries (less than 1MB font file).
  The storage position of Icon configuration file is 0x1D.

3.29 CRC check code generating function

The ex-factory default is non-checking mode. User can activate the CRC checking mode via 'SDW terminal tool box'. For users choosing the CRC checking mode, 2 bytes check codes are added to the command format. The command format is as following. The ex-factory setting for the display terminals are all non-checking mode.

[0xAA] [Command Code] [Data] [Check Code] [0xCC 0x33 0xC3 0x3C].

The 2 bytes check code is generated via the following function according to the '0xAA' 'Command Code' and 'Data'. Client needs to embed this function into his own program and get a 2 bytes check code which is then transferred to the display terminal.

After receiving the command, the display terminal will figure out a 2 bytes check code by calculation based on the contents of the command. The new check code calculated will be compared with the received check code to judge if the communication data have any error. If the comparing result is the same, then the display terminal will execute this display command. Otherwise, the display will fail to execute this display command.

```
//************************************
//function: CRC check
//input: frame head address, valid length
//output: CRC check value
//************************************
unsigned int crc_chk_value(unsigned char *data_value,unsigned char length) reentrant
{   unsigned int crc_value = 0xFFFF;
int i;
while(length--)
{ crc_value ^= *data_value++;
for(i=0;i<8;i++)
{ if(crc_value & 0x0001)
crc_value = (crc_value >>1) ^ 0xa001;
else
crc_value=crc_value >> 1;
}
}
return(crc_value);
}
```

Appendix 1: C51 Routine

```
////////////////////////////////////////////////////////////////////////////////////////////////
///Applicable for SDW series display terminal////////
///STC11F60XE、Crystal frequency 22.1184MHz、Baudrate115200bps  ///////
///Pls email support@viewtech.cn for code.                    ///////
////////////////////////////////////////////////////////////////////////////////////////////////

#include <reg51.h>
#define uchar unsigned char
#define uint unsigned int
void com_init( void )
{
SCON = 0x50;// mode 1，reception allowed
TMOD = 0x20; //T1 as Baud rate generator, work mode 2.
PCON = 0x80;//set Baud rate,11.0592MHz,19200bps
TH1 = 0xff;//
TL1 = 0xff;//
TR1 = 1;//activate T1
EA = 0; //close
```

```
ES = 0; // close the serial interface
}
//void com_interupt( void ) interrupt 4 //port interrupts the service program
//       {}
void transmit_byte ( uchar character )
{
SBUF = character;
while(~TI);
TI = 0;
}
//display string quickly
void quik_disp_str(uint x, uint y, uchar mode, uchar font, uint fcolor, uint bcolor, uchar *p, uint sum)
{
uint n;
transmit_byte( 0xaa );//command head
transmit_byte( 0x98 );//command code，display string quickly
transmit_byte( x/256 ); transmit_byte( x%256 ); //transmit X coordinate
transmit_byte( y/256 ); transmit_byte( y%256 ); // transmit Y coordinate
transmit_byte( mode );    transmit_byte( font );    //mode & font
transmit_byte( fcolor/256 ); transmit_byte( fcolor%256 );// transmit Fcolor
transmit_byte( bcolor/256 ); transmit_byte( bcolor%256 ); // transmit Bcolor
for( n=0;n<sum;n++ ){transmit_byte(*p); p++;}//send the data string
transmit_byte( 0xcc );//finish code
transmit_byte( 0x33 );
transmit_byte( 0xc3 );
transmit_byte( 0x3c );
}
main()
{
uchar string[20] = {0xCE,0xE4,0xBA,0xBA,0xD6,0xD0,0xCF,0xD4,0xBF,0xC6,
0xBC,0xBC,0xD3,0xD0,0xCF,0xDE,0xB9,0xAB,0xCB,0xBE}; //define string
com_init(    );
quik_disp_str(10,10,0,1,0xffff,0x0000,string,20);//display string
while(1);
}
```